第 4 部分（共 5 部分）· 解答

随机微积分，一步一步

第 4 部分：定价与数值方法。
全部练习的完整解答



Rates with Bip

随机微积分，一步一步

第 4 部分：连续时间定价与数值方法。练习解答

Rates with Bip

随机微积分，一步一步。第 4 部分：连续时间定价与数值方法。练习解答。

© 2026 Rates with Bip。本书采用知识共享 署名-非商业性使用-禁止演绎 4.0 国际许可协议（CC BY-NC-ND 4.0）：你可以在注明作者 Rates with Bip 的前提下，为非商业目的原样复制和分享本书。许可协议：creativecommons.org/licenses/by-nc-nd/4.0。 配套笔记本以 MIT 许可协议免费提供：github.com/RatesWithBip/curriculum。

本书译自英文原版 Stochastic Calculus, Step by Step, Part 4: Solutions。代码、公式和数字与英文版完全相同。

全课程使用同一组数字：股价 $S_0 = 100$，行权价 $K = 105$，无风险利率 $r = 3\%$（连续复利），波动率 $\sigma = 20\%$，期限 $T = 1$ 年；某一课需要股票在现实世界中的期望收益率时，取 $\mu = 8\%$。这些数字是为教学而虚构的，所有随机路径都用固定种子模拟。书中没有任何市场数据。

本册收录《随机微积分，一步一步》第 4 部分全部练习和自测题的完整解答。请先自己做每道练习，再看解答。

解答中的每个数字都由一个用断言自我检验的脚本计算；代码解答完整列出。

本书仅供学习之用，不构成任何金融、投资、法律或税务建议。

目录

如何使用这本解答	5
19. Black-Scholes 方程	6
20. Black-Scholes 公式	12
21. 希腊字母	17
22. 用数值方法检验公式	23
23. 三叉树	29
24. 实践中的 delta 对冲	34
25. Girsanov 定理	40
26. 风险中性定价	46
27. 计价单位变换	51
28. Feynman-Kac 定理	56
29. 有限差分	62
30. 选择方法	68

如何使用这本解答

本册收录《随机微积分，一步一步》第 4 部分《连续时间定价与数值方法》的完整解答：每道练习都完整解出，每一课末尾的自测题也给出答案。它应该在你自己尝试过每道题**之后**再读。

- **手算练习**（每课的前六道）完整解出。每个解答都从**思路**（打开局面的那一个想法）开始，然后分步骤展开，每一步都说明理由，最后以验证或解释结束。如果你卡住了，先只读思路，再自己试一试，然后才去读步骤。
- **编程练习**（每课的最后两道）给出完整的 Python 解答及其确切的输出。本册中的每个数字都由一个用断言自我检验的脚本生成，使用固定的随机种子，所以只要你用同样的种子，自己运行的结果应与书中显示的位数一致。
- **自测答案**很简短：一两句话，有数字的地方给出数字。

课程的数字始终不变：$S_0 = 100$，$K = 105$，$r = 3\%$（连续复利），$\sigma = 20\%$，$T = 1$ 年，以及 $\mu = 8\%$，所以 $\theta = (\mu - r)/\sigma = 0.25$。看涨期权价值 7.1281，看跌期权价值 9.0248。

第 4 部分中几乎每个手算解答都用到三招之一：让两个相等表达式的 dW 项和 dt 项分别**相等**；在正态积分中**配方**，把一个权重变成钟形曲线的平移；或者**利用鞅**，它的 dt 项为零，平均值保持不变。如果你的答案不同，先检查常见的嫌疑：在该用 r 的地方写了 μ，把 V_t 与 V_τ 混淆，忘了对行权价贴现，漏了 $\sqrt{T}$，以及在该写方差的地方写了标准差。

本册仅供学习之用，不构成金融建议。

第 19 课解答

Black-Scholes 方程

练习 19.1 至 19.8 以及第 19 课自测题的解答。

练习 19.1

思路。 数一数两个收益来源，然后用乘积法则贴现。

第 1 步：法则。 在 dt 内，Δ_t 股赚 $\Delta_t\,dS_t$。现金是剩下的部分 $X_t - \Delta_t S_t$，它赚 $r(X_t - \Delta_t S_t)\,dt$。没有其他东西流入或流出，所以 $dX_t = \Delta_t\,dS_t + r(X_t - \Delta_t S_t)\,dt$。

第 2 步：对 $e^{-rt}X_t$ 用乘积法则。 贴现因子没有 dW 部分，所以没有交叉项：

$$d(e^{-rt}X_t) = e^{-rt}\,dX_t - re^{-rt}X_t\,dt.$$

第 3 步：代入第 1 步。 含 rX_t 的项相消：

$$d(e^{-rt}X_t) = e^{-rt}\big(\Delta_t\,dS_t - r\Delta_t S_t\,dt\big).$$

第 4 步：识别。 由同样的乘积法则，$d(e^{-rt}S_t) = e^{-rt}(dS_t - rS_t\,dt)$。所以 $d(e^{-rt}X_t) = \Delta_t\,d(e^{-rt}S_t)$。

用文字说。 以今天的货币度量，你的财富只通过你在贴现股票上的头寸变化。现金一旦贴现，就是平的。

练习 19.2

思路。 泰勒展开到二阶，再用第 14 课的乘法表。

第 1 步：泰勒展开。

$$dV = V_t\,dt + V_S\,dS + \tfrac{1}{2}V_{SS}(dS)^2 + V_{tS}\,dt\,dS + \tfrac{1}{2}V_{tt}(dt)^2 + \dots$$

第 2 步：平方项。 由 $dS = \mu S\,dt + \sigma S\,dW$ 以及乘法表 $dW\,dW = dt$、$dW\,dt = 0$、$dt\,dt = 0$：

$$(dS)^2 = \sigma^2 S^2\,dt, \qquad dt\,dS = 0, \qquad (dt)^2 = 0.$$

更高阶的项更小，所以也消失。

第 3 步：合并。

$$dV = \left(V_t + \mu S V_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS}\right) dt + \sigma S V_S \, dW.$$

练习 19.3

思路。 对着期权卖空 V_S 股，随机部分就消失了。一个无风险的头寸必须赚取 r。

第 1 步。 在这一步中保持 V_S 不变，$d\Pi = dV - V_S \, dS$。

第 2 步：代入伊藤引理和 dS。

$$d\Pi = \left(V_t + \mu S V_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS}\right) dt + \sigma S V_S \, dW - V_S(\mu S \, dt + \sigma S \, dW).$$

dW 项相消，$\mu S V_S \, dt$ 项也相消：

$$d\Pi = \left(V_t + \tfrac{1}{2}\sigma^2 S^2 V_{SS}\right) dt.$$

第 3 步：没有风险，所以是现金利率。 如果 Π 赚得比 r 多，你就会借钱持有它；如果少，你就会卖空它并投资现金。无论哪种都有白捡的钱，所以
$d\Pi = r\Pi \, dt = r(V - V_S S) \, dt$。

第 4 步：让两者相等。 $V_t + \tfrac{1}{2}\sigma^2 S^2 V_{SS} = rV - rSV_S$，整理后就是 Black-Scholes 方程。√

练习 19.4

思路。 直接代入，而且方程是线性的。

$V = S$。$V_t = 0$，$V_S = 1$，$V_{SS} = 0$。左边：rS。右边：rS。√

$V = Ke^{-r(T-t)}$。$V_t = rKe^{-r(T-t)} = rV$；$V_S = V_{SS} = 0$。左边：rV。右边：rV。√

它们的差。 $S - Ke^{-r(T-t)}$ 在 T 时价值 $S_T - K$：一份以 K 买入的**远期合约**。用我们的数字，它今天价值 -1.8968，为负，因为行权价高于远期价格。

为什么它必须满足方程。 方程是线性的：每一项都是 V 的一个导数乘以一个不依赖于 V 的系数。所以解的和或差仍是解。从金融上说，一个由对冲后无风险头寸组成的组合，也是对冲后无风险的。

练习 19.5

思路。 代入并读出常数。

第 1 步：导数。 $V = S^2 e^{b(T-t)}$：$V_t = -bV$，$V_S = 2Se^{b(T-t)}$，$V_{SS} = 2e^{b(T-t)}$。所以 $SV_S = 2V$，$S^2 V_{SS} = 2V$。

第 2 步：代入。 $-bV + 2rV + \sigma^2 V = rV$。

第 3 步：除以 V。 $b = r + \sigma^2 = 0.07$。

在 $t = T$ 时，$V = S_T^2$，即收益。√ 所以这份合约价值

$$S_0^2 e^{(r+\sigma^2)T} = 10{,}000\, e^{0.07} = 10725.08.$$

验证。 作为风险中性平均值（第 17 课，取 $\mu = r$）：$e^{-rT} E[S_T^2] = e^{-rT} S_0^2 e^{(2r+\sigma^2)T}$，即 10725.08。√

练习 19.6

思路。 股息是股票上的额外收入，记入自融资法则。其他一切不变。

第 1 步：法则。 现在 $dX = \Delta\, dS + \Delta q S\, dt + r(X - \Delta S)\, dt$。股票本身的价格仍然服从 $dS = \mu S\, dt + \sigma S\, dW$。

第 2 步：代入 dS。

$$dX = \big(rX + \Delta(\mu + q - r)S\big)\, dt + \Delta\sigma S\, dW.$$

第 3 步：用伊藤引理让 dW 相等： $\Delta = V_S$，与前面相同。

第 4 步：让 dt 相等。 $rV + V_S(\mu + q - r)S = V_t + \mu S V_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS}$。$\mu S V_S$ 相消。整理：

$$V_t + (r - q)S V_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS} = rV.$$

用文字说。 方程中股票的增长率降为 $r - q$，因为它的一部分收益以现金形式到来。

练习 19.7

方程的残差，对看涨期权、看跌期权以及收益本身。

```
import numpy as np

Ncdf = np.vectorize(lambda x: 0.5 * (1 + math.erf(x / math.sqrt(2))))


def call(s, tau):
    dp = (np.log(s / K) + (r + 0.5 * sigma**2) * tau) / (sigma * np.sqrt(tau))
    return s * Ncdf(dp) - K * np.exp(-r * tau) * Ncdf(dp - sigma * np.sqrt(tau))


put = lambda s, tau: call(s, tau) - s + K * np.exp(-r * tau)      # put-call parity


def terms(f, s, tau, h=0.01, k=1e-5):
    Vt = (f(s, tau - k) - f(s, tau + k)) / (2 * k)                # t up means tau down
    Vs = (f(s + h, tau) - f(s - h, tau)) / (2 * h)
    Vss = (f(s + h, tau) - 2 * f(s, tau) + f(s - h, tau)) / h**2
    return Vt, r * s * Vs, 0.5 * sigma**2 * s**2 * Vss, r * f(s, tau)


def residual(f, s, tau):
    Vt, a, b, c = terms(f, s, tau)
    return Vt + a + b - c


worst = {"call": 0.0, "put": 0.0}
for s in (70.0, 85.0, 100.0, 115.0, 130.0):
    for tau in (0.25, 0.5, 1.0):
        worst["call"] = max(worst["call"], abs(residual(call, s, tau)))
        worst["put"] = max(worst["put"], abs(residual(put, s, tau)))
print(f"largest residual: call {worst['call']:.1e}, put {worst['put']:.1e}")

payoff = lambda s, tau: max(s - K, 0.0)
print("payoff shape:      V_t   rSV_S  (1/2)s^2S^2V_SS     rV   residual")
for s in (100.0, 110.0):
    Vt, a, b, c = terms(payoff, s, 0.5)
    print(f"  S = {s:5.1f}   {Vt:6.2f}  {a:6.2f}  {b:15.2f}  {c:5.2f}  {Vt + a + b -
c:9.2f}")
```

它的输出。

```
largest residual: call 5.5e-07, put 5.4e-07
payoff shape:      V_t   rSV_S  (1/2)s^2S^2V_SS     rV   residual
  S = 100.0     0.00    0.00            0.00   0.00       0.00
  S = 110.0     0.00    3.30            0.00   0.15       3.15
```

读输出。 看涨和看跌期权都满足方程，误差只是扰动的舍入。收益形状的函数则不满足。在 $S = 110$ 处它有 $V_S = 1$，且不依赖于时间，所以左边是 rS，而右边是 $r(S - K)$。差距恰好是 $rK = 3.15$：缺少的是 V_t 项。在 $S = 100$ 处每一项都是零，方

程成立，但这个函数在行权价处有一个折点，那里 V_{SS} 不存在。真正的价格必须从行权价以上的区域平滑地过渡到以下的区域，而正是 V_t 使这成为可能。

练习 19.8

三种步长下对冲后的变化。

```
rng = np.random.default_rng(198)
Z = rng.standard_normal(400_000)
V0 = float(call(S0, T))
delta = float((call(S0 + 0.01, T) - call(S0 - 0.01, T)) / 0.02)
print(" h (days)   sd of hedged change")
prev, ratios = None, []
for days in (16, 4, 1):
    h = days / 252
    S1 = S0 * np.exp((mu - 0.5 * sigma**2) * h + sigma * math.sqrt(h) * Z)
    change = call(S1, T - h) - V0 - delta * (S1 - S0)
    sd = change.std()
    ratios.append(float(prev / sd) if prev else 0.0)
    prev = sd
    print(f"{days:9d}   {sd:19.5f}")
print("shrink factors:", [round(x, 2) for x in ratios[1:]])
```

它的输出。

```
 h (days)   sd of hedged change
       16               0.36348
        4               0.08990
        1               0.02241
shrink factors: [4.04, 4.01]
```

读输出。 h 每除以 4，分散程度大约缩小到四分之一：它与 h 成正比。原因在于泰勒展开的余项：对冲消去 $V_S\,\delta S$ 之后，剩下的主要是 $\frac{1}{2}V_{SS}(\delta S)^2$，它与 h 乘以一个随机的 Z^2 成正比。对比单独的期权，它的分散程度与 $\sqrt{h}$ 成正比。当 h 趋于 0 时，两者之比趋于零，这就是为什么对冲在极限中是精确的。

自测

1. V_S：期权价值对股价的斜率。

2. 因为它被消掉了。它出现在期权的漂移中（$\mu S V_S$），也以完全相同的形式出现在对冲的漂移中，而两者被匹配了。

3. 时间衰减 $V_t = -5.28$，以及凸性项 $\frac{1}{2}\sigma^2 S^2 V_{SS} = 3.99$。它们与 rSV_S 之和必须等于很小的数 rV，因为对冲后的头寸只赚取现金利率：它从凸性中赚到的，必须以时间衰减偿还。

第 20 课解答

Black-Scholes 公式

练习 20.1 至 20.8 以及第 20 课自测题的解答。

练习 20.1

思路。 相除，取对数，解出 Z。

第 1 步。 $S_0 e^{(r-\frac{1}{2}\sigma^2)T+\sigma\sqrt{T}Z} > K \iff (r-\frac{1}{2}\sigma^2)T+\sigma\sqrt{T}Z > \ln(K/S_0)$。（取对数保持不等号，因为 ln 是增函数。）

第 2 步。 利用 $\ln(K/S_0) = -\ln(S_0/K)$，并除以 $\sigma\sqrt{T} > 0$：

$$Z > -\frac{\ln(S_0/K)+(r-\frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}} = -d_-.$$

数字。 分子是 $\ln(100/105)+0.01 = -0.04879+0.01 = -0.03879$，分母是 0.2。所以 $d_- =$ -0.19395，临界 Z 值为 0.19395。

练习 20.2

思路。 直接套公式，先算出贴现的行权价。

- **股票项：** $S_0 N(d_+) = 100 \times 0.50241 = 50.241$。
- **贴现的行权价：** $105e^{-0.03} = 101.897$。
- **行权价项：** $101.897 \times 0.42311 = 43.114$。
- **看涨期权：** $50.241 - 43.114 = 7.127$。

第一项是对冲中投在股票上的钱。第二项是为此借入的钱。

练习 20.3

思路。 正态密度乘以指数函数，是一个平移的正态密度。

第 1 步：积分。 取 $a = \sigma\sqrt{T}$，在 $z > -d_-$ 上：

$$e^{-rT}\tilde{E}[S_T \mathbf{1}\{S_T > K\}] = e^{-rT}\int_{-d_-}^{\infty} S_0 e^{(r-\frac{1}{2}\sigma^2)T + az}\varphi(z)\,dz = S_0 e^{-\frac{1}{2}a^2}\int_{-d_-}^{\infty} e^{az}\varphi(z)\,dz.$$

第 2 步：配方。 $az - \frac{1}{2}z^2 = -\frac{1}{2}(z-a)^2 + \frac{1}{2}a^2$，所以 $e^{az}\varphi(z) = e^{\frac{1}{2}a^2}\varphi(z-a)$。

第 3 步：相消。 两个常数是 $e^{-\frac{1}{2}a^2}$ 和 $e^{\frac{1}{2}a^2}$：

$$= S_0\int_{-d_-}^{\infty}\varphi(z-a)\,dz.$$

第 4 步：代换 $u = z - a$：

$$= S_0\int_{-d_- - a}^{\infty}\varphi(u)\,du = S_0 P(Z > -d_- - a) = S_0 N(d_- + a) = S_0 N(d_+).$$

练习 20.4

思路。 看跌期权在 $Z < -d_-$ 时支付。用同样的两项，只是在另一侧。

第 1 步：拆分。 $P = Ke^{-rT}\tilde{P}(S_T < K) - e^{-rT}\tilde{E}[S_T\mathbf{1}\{S_T < K\}]$。

第 2 步：行权价项。 $\tilde{P}(S_T < K) = P(Z < -d_-) = N(-d_-)$。

第 3 步：股票项。 与练习 20.3 相同，但从 $-\infty$ 积分到 $-d_-$：

$$S_0\int_{-\infty}^{-d_-}\varphi(z-a)\,dz = S_0\int_{-\infty}^{-d_- - a}\varphi(u)\,du = S_0 N(-d_+).$$

所以 $P = Ke^{-rT}N(-d_-) - S_0 N(-d_+) = 9.0248$。

平价。 $N(-x) = 1 - N(x)$，所以

$$P = Ke^{-rT}(1 - N(d_-)) - S_0(1 - N(d_+)) = C - S_0 + Ke^{-rT}.$$

√ 数值上，$C - S_0 + Ke^{-rT} = 9.0242$（使用练习 20.2 中的舍入值）。

练习 20.5

思路。 这就是公式的两项，分开出售。

- **现金数字期权：** $e^{-rT}\tilde{P}(S_T > K) = e^{-rT}N(d_-) = 0.4106$。
- **资产数字期权：** $e^{-rT}\tilde{E}[S_T\mathbf{1}\{S_T > K\}] = S_0 N(d_+) = 50.2413$。

看涨期权。 若 $S_T > K$：一份资产数字期权支付 S_T，K 份现金数字期权支付 K，所以差额支付 $S_T - K$。否则一切都支付 0。所以看涨期权是一份资产数字期权减去 K 份现金数字期权：$50.2413 - 105 \times 0.4106 = 7.1281$。√

练习 20.6

思路。 注意 $d_\pm$ 中 $\ln(S_0/K) + rT$ 的符号，因为分母趋于 0。

(a) $\sigma \to 0$。 $d_\pm \to \pm\infty$，取决于 $S_0 - Ke^{-rT}$ 的符号。若为正，两个 N 都趋于 1，$C \to S_0 - Ke^{-rT}$。若为负，两者都趋于 0，$C \to 0$。所以 $C \to \max(S_0 - Ke^{-rT}, 0)$。没有随机性时，股价以 r 增长，你今天就知道结果。用我们的数字，$Ke^{-rT} = 101.90$，高于股价，所以极限为 0。

(b) $T \to 0$。 类似地，$C \to \max(S_0 - K, 0)$：就是收益本身。

(c) $S_0 \to \infty$。 两个 $d_\pm \to \infty$，所以 $C \to S_0 - Ke^{-rT}$。看涨期权必定被行权，所以它与远期合约等值：一股减去贴现的行权价。

练习 20.7

对收益乘以密度用梯形法则。各列是相对于公式的误差。

```
import numpy as np


def formula(k):
    d2 = (math.log(S0 / k) + (r - 0.5 * sigma**2) * T) / (sigma * math.sqrt(T))
    return S0 * N(d2 + sigma * math.sqrt(T)) - k * disc * N(d2)


def integrate(k, points):
    # Trapezium rule on payoff times density, z from -8 to 8.
    z = np.linspace(-8, 8, points)
    s_t = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * z)
    y = disc * np.maximum(s_t - k, 0) * np.exp(-z**2 / 2) / math.sqrt(2 * math.pi)
    return float(((y[1:] + y[:-1]) / 2 * np.diff(z)).sum())


print(" strike   formula      100 pts     1000 pts    10000 pts")
for k in (85, 95, 105, 115, 125):
    print(f"{k:7d}  {formula(k):8.4f}  " + "  ".join(f"{integrate(k, p) - formula(k):+10.6f}"
for p in (100, 1000, 10000)))
```

它的输出。

```
 strike   formula      100 pts     1000 pts    10000 pts
     85   19.1313   -0.001521   +0.000037   +0.000000
     95   12.1797   +0.006564   +0.000059   -0.000001
    105    7.1281   +0.004564   +0.000073   +0.000000
    115    3.8596   +0.007865   -0.000145   -0.000001
    125    1.9521   -0.006090   -0.000090   +0.000001
```

读输出。 在 $[-8, 8]$ 上取 1000 个点，间距 0.016，最坏误差为 1.5e-04，视行权价而定，精确到小数点后 3 到 4 位。取 10,000 个点，精确到约 6 位。误差并不完全平稳，原因与树相同：对每个行权价，$S_T = K$ 处的折点落在网格点之间的不同位置。像笔记本那样把一个网格点放在折点上并从那里开始积分，就能消除这一点。

练习 20.8

用蒙特卡洛计算数字期权和看涨期权，附 95% 半宽。

```
rng = np.random.default_rng(208)
M = 1_000_000
Z = rng.standard_normal(M)
S_T = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)
print("           estimate   95% half-width   exact      half-width/price")
for name, y, exact in (("digital", disc * (S_T > K), disc * N(dm)), ("call", disc *
np.maximum(S_T - K, 0), S0 * N(dp) - K * disc * N(dm))):
    est, hw = y.mean(), 1.96 * y.std(ddof=1) / math.sqrt(M)
    print(f"{name:9s} {est:9.5f}   {hw:14.5f}   {exact:7.5f}   {hw / est:12.3%}")
```

它的输出。

```
           estimate   95% half-width   exact      half-width/price
digital     0.41124          0.00094   0.41060         0.229%
call        7.12716          0.02457   7.12806         0.345%
```

读输出。 看涨期权的半宽更大，无论是绝对值还是相对于其价格。相对误差由单个收益的标准差与其均值之比决定：数字期权为 1.17，看涨期权为 1.76。数字期权至多支付 1，所以它的收益分散程度有限。看涨期权有一条长长的右尾：罕见的、极大的收益贡献了它的大部分方差。

自测

1. $N(d_-) = 0.4231$，约 42%。

2. 它把钟形曲线向右平移 $\sigma\sqrt{T}$，而不改变形状（除了一个常数因子）。

3. 50.24 和 43.11。两者合起来就是对冲：大约半股，由一笔贷款融资。

第 21 课解答

希腊字母

练习 21.1 至 21.8 以及第 21 课自测题的解答。

练习 21.1

思路。 利用 d_- 的定义，用 $\varphi(d_-)$ 表示 $\varphi(d_+)$。

第 1 步。 $d_+^2 = (d_- + \sigma\sqrt{\tau})^2 = d_-^2 + 2d_-\sigma\sqrt{\tau} + \sigma^2\tau$。

第 2 步。 所以 $\varphi(d_+) = \varphi(d_-)\exp\left(-d_-\sigma\sqrt{\tau} - \tfrac{1}{2}\sigma^2\tau\right)$。

第 3 步。 由 d_- 的定义：$d_-\sigma\sqrt{\tau} = \ln(S/K) + r\tau - \tfrac{1}{2}\sigma^2\tau$。代入后，$\tfrac{1}{2}\sigma^2\tau$ 项相消：

$$\varphi(d_+) = \varphi(d_-)\exp\left(-\ln(S/K) - r\tau\right) = \varphi(d_-)\frac{K}{S}e^{-r\tau}.$$

第 4 步。 乘以 S：$S\varphi(d_+) = Ke^{-r\tau}\varphi(d_-)$。∎

练习 21.2

思路。 乘积法则、链式法则，然后恒等式让多出的项消失。

第 1 步：$d_\pm$ 的导数。 只有 $\ln S$ 依赖于 S，所以 $\dfrac{\partial d_\pm}{\partial S} = \dfrac{1}{S\sigma\sqrt{\tau}}$。

第 2 步：求导。 利用 $N' = \varphi$：

$$\frac{\partial C}{\partial S} = N(d_+) + S\varphi(d_+)\frac{1}{S\sigma\sqrt{\tau}} - Ke^{-r\tau}\varphi(d_-)\frac{1}{S\sigma\sqrt{\tau}}.$$

第 3 步：用恒等式。 最后两项大小相等、符号相反。所以 $\Delta = N(d_+)$。

练习 21.3

思路。 再用一次链式法则。

$$\Gamma = \frac{\partial}{\partial S} N(d_+) = \varphi(d_+)\frac{\partial d_+}{\partial S} = \frac{\varphi(d_+)}{S\sigma\sqrt{\tau}}.$$

在 $S = 100$、$\tau = 1$ **处。** $\varphi(d_+) =$ 0.39893，所以 $\Gamma = 0.39893/(100 \times 0.2) =$ 0.01995。

效果。 股价上涨 1，delta 约上升 $\Gamma \times 1 = 0.020$，从 0.502 升到约 0.522。

练习 21.4

思路。 求导，对 $\partial d_\pm/\partial t$ 部分用恒等式。剩下的是两项。

第 1 步。 由 $\tau = T - t$，贴现因子给出 $\dfrac{\partial}{\partial t} e^{-r\tau} = re^{-r\tau}$。所以

$$\frac{\partial C}{\partial t} = S\varphi(d_+)\frac{\partial d_+}{\partial t} - Ke^{-r\tau}\varphi(d_-)\frac{\partial d_-}{\partial t} - rKe^{-r\tau}N(d_-).$$

第 2 步：恒等式。 它把 $Ke^{-r\tau}\varphi(d_-)$ 换成 $S\varphi(d_+)$，所以前两项变成 $S\varphi(d_+)\dfrac{\partial}{\partial t}(d_+ - d_-)$。

第 3 步：差。 $d_+ - d_- = \sigma\sqrt{\tau}$，所以 $\dfrac{\partial}{\partial t}(\sigma\sqrt{T-t}) = -\dfrac{\sigma}{2\sqrt{\tau}}$。因此

$$\Theta = -\frac{S\varphi(d_+)\sigma}{2\sqrt{\tau}} - rKe^{-r\tau}N(d_-).$$

第 4 步：方程。 利用 $\frac{1}{2}\sigma^2 S^2\Gamma = \frac{1}{2}\sigma^2 S^2 \dfrac{\varphi(d_+)}{S\sigma\sqrt{\tau}} = \dfrac{S\varphi(d_+)\sigma}{2\sqrt{\tau}}$。所以 Θ 的第一项与 Γ 项相消：

$$\begin{aligned}\Theta + rS\Delta + \tfrac{1}{2}\sigma^2 S^2\Gamma &= -rKe^{-r\tau}N(d_-) + rSN(d_+) \\ &= r\big(SN(d_+) - Ke^{-r\tau}N(d_-)\big) = rC.\end{aligned}$$

√

练习 21.5

思路。 $P = C - S + Ke^{-r\tau}$，所以对三项分别求导。

- **Delta：** $\Delta_P = \Delta_C - 1 =$ -0.4976。
- **Gamma：** $-S$ 和 $Ke^{-r\tau}$ 两项是 S 的直线，没有二阶导数。所以 $\Gamma_P = \Gamma_C =$ 0.01995。
- **Theta：** $\Theta_P = \Theta_C + rKe^{-r\tau} =$ -5.2827 + 3.0569 = -2.2258。

哪些相同，为什么。 Gamma 相同，vega 也相同：看涨与看跌期权之差是一份远期合约，它的价值是 S 的线性函数，且不依赖于 σ。两个期权带有同样的凸性。Theta 不同，因为看跌期权持有对贴现行权价的索取权，而它随时间增长。

练习 21.6

思路。 还是同样的模式：只有 $d_{\pm}$ 中的 σ 要紧，而恒等式把它化简了。

第 1 步。 d_+ 和 d_- 都依赖于 σ，且 $d_+ - d_- = \sigma\sqrt{\tau}$。

$$\frac{\partial C}{\partial \sigma} = S\varphi(d_+)\frac{\partial d_+}{\partial \sigma} - Ke^{-r\tau}\varphi(d_-)\frac{\partial d_-}{\partial \sigma} = S\varphi(d_+)\frac{\partial (d_+ - d_-)}{\partial \sigma} = S\varphi(d_+)\sqrt{\tau}.$$

第 2 步。 由 $\Gamma = \varphi(d_+)/(S\sigma\sqrt{\tau})$，得 $\varphi(d_+) = \Gamma S\sigma\sqrt{\tau}$，所以 vega 是 $S \cdot \Gamma S\sigma\sqrt{\tau} \cdot \sqrt{\tau} = \sigma\tau S^2\Gamma$。

数字。 两种方法都得到 39.89。波动率上升一个点，价格增加 0.40。

为什么关系如此密切。 波动率只通过平方移动的期望大小 $\sigma^2\tau$ 进入价格。而把平方移动变成钱的正是 gamma。更多的波动率和更多的凸性做的是同一件事。

练习 21.7

扰动与公式的对比，然后是 gamma 误差随扰动大小的变化。

```
import numpy as np

phi = lambda x: math.exp(-x**2 / 2) / math.sqrt(2 * math.pi)


def d1(s, tau, sig=sigma):
    return (math.log(s / K) + (r + 0.5 * sig**2) * tau) / (sig * math.sqrt(tau))


def call(s, tau, sig=sigma):
    dp = d1(s, tau, sig)
    return s * Nf(dp) - K * math.exp(-r * tau) * Nf(dp - sig * math.sqrt(tau))


def formulas(s, tau=T):
    dp = d1(s, tau); dm = dp - sigma * math.sqrt(tau)
    return {"delta": Nf(dp), "gamma": phi(dp) / (s * sigma * math.sqrt(tau)),
            "theta": -s * phi(dp) * sigma / (2 * math.sqrt(tau)) - r * K * math.exp(-r * tau)
* Nf(dm), "vega": s * phi(dp) * math.sqrt(tau)}


def bumps(s, tau=T, h=0.01, k=1e-5, v=1e-4):
    return {"delta": (call(s + h, tau) - call(s - h, tau)) / (2 * h), "gamma": (call(s + h,
tau) - 2 * call(s, tau) + call(s - h, tau)) / h**2,
            "theta": (call(s, tau - k) - call(s, tau + k)) / (2 * k), "vega": (call(s, tau,
sigma + v) - call(s, tau, sigma - v)) / (2 * v)}


print("   S   greek      formula         bump")
for s in (80.0, 100.0, 120.0):
    f_, b_ = formulas(s), bumps(s)
    for g in ("delta", "gamma", "theta", "vega"):
        print(f"{s:4.0f}   {g:6s} {f_[g]:11.6f}  {b_[g]:11.6f}")
print("gamma error at S = 100, by bump size:")
exact = formulas(100.0)["gamma"]
errs = {}
for h in (1.0, 0.1, 0.01, 0.001, 0.0001):
    errs[h] = bumps(100.0, h=h)["gamma"] - exact
    print(f"  h = {h:<7g} {errs[h]:+.2e}")
```

它的输出。

```
   S   greek      formula         bump
  80   delta     0.133571     0.133571
  80   gamma     0.013471     0.013471
  80   theta    -2.015176    -2.015176
  80   vega     17.242994    17.242993
 100   delta     0.502413     0.502413
 100   gamma     0.019947     0.019947
 100   theta    -5.282748    -5.282748
 100   vega     39.893498    39.893498
 120   delta     0.820601     0.820601
 120   gamma     0.010910     0.010910
 120   theta    -5.476186    -5.476186
 120   vega     31.421930    31.421928
gamma error at S = 100, by bump size:
  h = 1       -3.81e-06
  h = 0.1     -3.81e-08
  h = 0.01    -3.86e-10
  h = 0.001   -2.59e-09
  h = 0.0001  +3.17e-07
```

读输出。 扰动结果与公式相符。扰动每缩小到十分之一，gamma 误差就缩小到百分之一，这是第 29 课的 h^2 误差，直到大约 $h = 0.01$。然后误差又上升了。那是舍入：二阶差分把一个只精确到约 10^{-14} 的极小的数除以 h^2。所以对小的 h，误差像 $10^{-14}/h^2$ 那样增长。存在一个最佳的扰动大小，而对二阶导数来说它并不很小。

练习 21.8

对冲后做多看涨期权的精确一天利润，与 gamma 公式对比。

```
dt = 1 / 252
V, D, G = call(100.0, T), formulas(100.0)["delta"], formulas(100.0)["gamma"]
print("  move     exact P&L    gamma formula")
pl = {}
for m in (0.0, 0.005, -0.005, 0.0126, -0.0126, 0.025, -0.025):
    s1 = 100.0 * (1 + m)
    exact_pl = call(s1, T - dt) - V - D * (s1 - 100.0) - r * (V - D * 100.0) * dt
    approx = 0.5 * G * 100.0**2 * (m**2 - sigma**2 * dt)
    pl[m] = exact_pl
    print(f"{m:+7.2%}   {exact_pl:9.4f}   {approx:12.4f}")
print(f"break-even move: sigma sqrt(dt) = {sigma * math.sqrt(dt):.2%}")
```

它的输出。

```
  move     exact P&L    gamma formula
 +0.00%     -0.0158        -0.0158
 +0.50%     -0.0135        -0.0133
 -0.50%     -0.0131        -0.0133
 +1.26%     -0.0005         0.0000
 -1.26%      0.0006         0.0000
 +2.50%      0.0450         0.0465
 -2.50%      0.0480         0.0465
break-even move: sigma sqrt(dt) = 1.26%
```

读输出。 公式很接近，随着变动缩小而更准。股价不变时，持有者每天损失 0.016。约 $\pm 1.26\%$ 的变动恰好持平。更大的变动，无论方向，都赚钱。上涨与下跌之间的小差别是公式忽略的三阶项。

自测

1. $\Delta = 0.5024$，$\Gamma = 0.01995$。

2. 看涨期权减去看跌期权是一份远期合约，其价值是 S 的一条直线。直线没有曲率，所以两个期权的 gamma 相同。

3. 他们赔钱。Theta 为负：没有变动时，gamma 什么也赚不到，而等待的代价仍要支付。

第 22 课解答

用数值方法检验公式

练习 22.1 至 22.8 以及第 22 课自测题的解答。

练习 22.1

思路。 由 $1.96s/\sqrt{M} = h$ 解出 M。

$M = (1.96s/h)^2$。取 $s = 12.5$：

- $h = 0.01$：$M = 6{,}002{,}500$ 条路径。
- $h = 0.001$：$M = 600{,}250{,}000$ 条路径。

为什么是 100 倍而不是 10 倍。 误差像 $1/\sqrt{M}$ 那样下降，所以 M 像 $1/h^2$ 那样增长。误差缩小到十分之一，要花一百倍的路径。

练习 22.2

思路。 两个相关量平均值的方差。

第 1 步。 Z 和 $-Z$ 同分布，所以 $f(Z)$ 和 $f(-Z)$ 的方差相同，记为 v。它们的协方差为 ρv。

第 2 步。

$$\operatorname{Var}\Big(\tfrac{1}{2}\big(f(Z) + f(-Z)\big)\Big) = \tfrac{1}{4}(v + v + 2\rho v) = \tfrac{1}{2}v(1 + \rho).$$

为什么这有帮助。 一对用了两个收益，所以要与两个独立抽样的平均比较，其方差为 $\tfrac{1}{2}v$。当 $\rho < 0$ 时对偶法更好。对单调的收益，$f(Z)$ 和 $f(-Z)$ 朝相反方向变动：一个高时另一个低。所以总有 $\rho < 0$，这个方法不会有害。（对跨式这样在两端都高的收益，ρ 可能为正，那时它就有害了。）

练习 22.3

思路。 折点落在节点之间，而落在哪里一直在变。

看涨期权。 最终节点之间以因子 u^2 间隔。实际上，树把有折点的收益替换成某种取决于 K 在节点之间位置的东西。n 每变化一，所有节点都会移动，所以 K 在其间隙中的位置改变，折点的"取整"方式也随之改变。因此误差上下跳动，只有它的包络线像 $1/n$ 那样缩小。

看跌期权。 它们在同一个 K 处有同样的折点。事实上，在同一棵树上，只要选择的 u、d 和 $\tilde{p}$ 使贴现股价成为鞅，看跌-看涨平价就精确成立：看涨和看跌期权的误差完全相同。

光滑的收益。 没有折点，像 S^2 这样的收益不在乎节点落在哪里。树对光滑函数的平均，其误差像 $1/n$ 那样平稳地缩小，没有跳动。

练习 22.4

思路。 关于未知数 c 的两个方程。

第 1 步。 若 $V_n = V + c/n$，$V_{2n} = V + c/(2n)$，则

$$2V_{2n} - V_n = 2V + \frac{c}{n} - V - \frac{c}{n} = V.$$

第 2 步。 $2 \times 9.0292 - 9.0330 = 9.0254$，对照公式的 9.0248。原始误差为 0.0082 和 0.0044；外推把误差缩小到 0.0006。

练习 22.5

思路。 加上一个均值为零的东西，并选择能抵消最多噪声的倍数。

无偏。 $E[\hat{V}] = E[Y] - b(E[X] - S_0) = E[Y]$，因为 $E[X] = S_0$。

它的方差。 对一条路径，$\mathrm{Var}(Y - bX) = \mathrm{Var}(Y) - 2b\,\mathrm{Cov}(Y, X) + b^2\mathrm{Var}(X)$。

最优的 b。 这是 b 的二次函数。令其导数为零，得到 $b^* = \mathrm{Cov}(Y, X)/\mathrm{Var}(X)$。

最小值。 代入：

$$\mathrm{Var}(Y) - \frac{\mathrm{Cov}(Y, X)^2}{\mathrm{Var}(X)} = \mathrm{Var}(Y)(1 - \rho^2).$$

所以 0.9 的相关系数消除了 81% 的方差：用五分之一的路径达到同样的精度。

练习 22.6

思路。 数字期权是一次缩放的抛硬币，其方差为 $p(1-p)$。

单个收益。 $s = e^{-rT}\sqrt{p(1-p)} = 0.97045 \times \sqrt{0.4231 \times 0.5769} = 0.4794$。

± 0.001 **所需的路径数**。 $(1.96 s / 0.001)^2 = 883{,}076$。

当 $p = 0.01$ **时**。 $s = 0.0966$，所以 ± 0.001 只需 35,817 条路径。

但这具有误导性。 更罕见的那个数字期权只值 0.0097，所以 ± 0.001 是 10% 的误差。要使第一个数字期权（价值 0.4106）达到 10% 的精度，只需 524 条路径。罕见事件在相对意义上是困难的：很少有路径能见到它们。

练习 22.7

普通、对偶和控制变量估计量，在三个行权价上。

```
import numpy as np

rng = np.random.default_rng(227)
M = 100_000
Z = rng.standard_normal(M)
grow = lambda z: S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * z)


def estimators(k):
    pay = lambda z: disc * np.maximum(grow(z) - k, 0)
    plain = pay(Z)
    # The antithetic estimator uses two payoffs per draw: compare at equal numbers of
payoffs.
    anti = 0.5 * (pay(Z[: M // 2]) + pay(-Z[: M // 2]))
    x = disc * grow(Z)                                      # known mean: S0
    b = np.cov(plain, x)[0, 1] / x.var(ddof=1)
    ctrl = plain - b * (x - S0)
    se_p, se_a, se_c = plain.std(ddof=1) / math.sqrt(M), anti.std(ddof=1) / math.sqrt(M //
2), ctrl.std(ddof=1) / math.sqrt(M)
    return plain.mean(), se_p, anti.mean(), se_a, ctrl.mean(), se_c, np.corrcoef(plain, x)[0,
1]


print(" strike   plain (se)         antithetic (se)    control (se)       corr    worth
(anti, ctrl)")
results = {}
for k in (85.0, 105.0, 125.0):
    p_, sp, a_, sa, c_, sc, rho = estimators(k)
    results[k] = (sp / sa) ** 2, (sp / sc) ** 2
    print(f"{k:7.0f}  {p_:7.4f} ({sp:.4f})   {a_:7.4f} ({sa:.4f})   {c_:7.4f} ({sc:.4f})
{rho:.3f}   {(sp / sa) ** 2:5.1f} x, {(sp / sc) ** 2:5.1f} x")
```

它的输出。

```
 strike   plain (se)         antithetic (se)    control (se)       corr    worth (anti, ctrl)
     85  19.1106 (0.0572)   19.1186 (0.0253)   19.1172 (0.0113)   0.980     5.1 x,  25.6 x
    105   7.1170 (0.0396)    7.1155 (0.0325)    7.1211 (0.0192)   0.875     1.5 x,   4.3 x
    125   1.9415 (0.0213)    1.9418 (0.0204)    1.9432 (0.0159)   0.666     1.1 x,   1.8 x
```

读输出。 在 $K = 85$ 时，看涨期权的行为几乎与股票一样（相关系数 0.98），控制变量相当于普通路径的 26 倍。在 $K = 125$ 时，大多数路径不支付任何东西，相关系数较低，它只相当于 1.8 倍。这个倍数是 $1/(1-\rho^2)$。对偶法的帮助也类似：在低行权价处最大，那里的收益最接近 Z 的直线，而在高行权价处几乎没有帮助。

练习 22.8

美式看跌期权：普通、相邻平均、外推，以及行权边界。

```
def american(n, s0=S0):
    dt = T / n
    u = math.exp(sigma * math.sqrt(dt)); d = 1 / u; p_ = (math.exp(r * dt) - d) / (u - d); df
= math.exp(-r * dt)
    v = np.maximum(K - s0 * u ** (2 * np.arange(n + 1) - n), 0)
    for i in range(n - 1, -1, -1):
        v = df * (p_ * v[1:] + (1 - p_) * v[:-1])
        v = np.maximum(v, K - s0 * u ** (2 * np.arange(i + 1) - i))
    return float(v[0])


avg = lambda n, s0=S0: 0.5 * (american(n, s0) + american(n + 1, s0))
bench = avg(6400)
print(f"benchmark (n = 6400): {bench:.5f}")
print("      n   plain err   averaged err   Richardson err")
errs = {}
for n in (50, 100, 200, 400, 800):
    errs[n] = (avg(n) - bench, 2 * avg(2 * n) - avg(n) - bench)
    print(f"{n:7d}   {american(n) - bench:+9.5f}   {errs[n][0]:+12.5f}   {errs[n]
[1]:+14.5f}")

# Below the boundary the option is worth exactly its exercise value.
lo, hi = 50.0, 100.0
for _ in range(30):
    mid = 0.5 * (lo + hi)
    if american(800, mid) - (K - mid) > 1e-6:
        hi = mid
    else:
        lo = mid
print(f"exercise now if S is below about {hi:.2f}")
```

它的输出。

```
benchmark (n = 6400): 9.47747
      n   plain err   averaged err   Richardson err
     50    -0.00143       +0.01529         +0.00421
    100    +0.01003       +0.00975         +0.00038
    200    +0.00501       +0.00506         -0.00234
    400    +0.00420       +0.00136         -0.00021
    800    +0.00203       +0.00058         +0.00018
exercise now if S is below about 81.17
```

读输出。 相邻平均整理了普通树的跳动。Richardson 外推通常做得更好，但并不可靠，因为平均后的序列并不完全平稳。到 800 步时，它与基准相差约 0.0002 以内。边界：在剩余一年时，如果股价低于约 81.2，就立即行权。低于这个水平时，行权价上的利息比期权剩余的灵活性更值钱。

自测

1. 100 倍。

2. 因为误差随着行权价在节点之间的位置而跳动，所以某一个大小可能碰巧很准。要用一系列大小上的最坏误差。

3. 它比较子节点的贴现平均与行权的价值，保留较大的那个。

第 23 课解答

三叉树

练习 23.1 至 23.8 以及第 23 课自测题的解答。

练习 23.1

思路。 方差方程给出 $p_u + p_d$，均值方程给出 $p_u - p_d$。相加、相减即可。

第 1 步。 $(p_u + p_d)\Delta x^2 = \sigma^2 \Delta t$，且 $\Delta x^2 = \lambda^2 \sigma^2 \Delta t$，给出 $p_u + p_d = 1/\lambda^2$。

第 2 步。 概率之和为 1，所以 $p_m = 1 - 1/\lambda^2$。

第 3 步。 $(p_u - p_d)\Delta x = \nu \Delta t$ 给出 $p_u - p_d = \nu \Delta t / \Delta x$。

第 4 步。 相加、相减：

$$p_u = \frac{1}{2}\left(\frac{1}{\lambda^2} + \frac{\nu \Delta t}{\Delta x}\right), \qquad p_d = \frac{1}{2}\left(\frac{1}{\lambda^2} - \frac{\nu \Delta t}{\Delta x}\right).$$

数字。 $\Delta x = \sqrt{3} \times 0.2 \times 0.1 = \ 0.03464$，倾斜量 $\nu \Delta t / \Delta x = 0.0001 / \ 0.03464 \ = 0.00289$。所以 $p_u = 0.16811$，$p_m = 0.66667$，$p_d = 0.16522$。

练习 23.2

思路。 奇数阶矩只依赖于 $p_u - p_d$。

第 1 步：在树上。 $E[\Delta X^3] = (p_u - p_d)\Delta x^3 = \dfrac{\nu \Delta t}{\Delta x}\Delta x^3 = \nu \Delta t \, \Delta x^2 = \lambda^2 \nu \sigma^2 \Delta t^2$。

第 2 步：对正态分布。 $m^3 + 3ms^2 = \nu^3 \Delta t^3 + 3\nu\sigma^2 \Delta t^2$。在首阶上是第二项：$3\nu\sigma^2\Delta t^2$。

第 3 步：两者一致，当 $\lambda^2 = 3$。 所以 $\lambda = \sqrt{3}$ 不仅匹配四阶矩，也匹配三阶矩。取 $n = 100$，两者都是 $3 \times 0.01 \times 0.04 \times 0.0001 = 1.2 \times 10^{-7}$。

练习 23.3

思路。 偶数阶矩只依赖于 $p_u + p_d$。

树。 $E[\Delta X^4] = (p_u + p_d)\Delta x^4 = \dfrac{1}{\lambda^2}\lambda^4\sigma^4\Delta t^2 = \lambda^2\sigma^4\Delta t^2$。

正态分布。 方差为 $s^2 = \sigma^2\Delta t$，在这一阶上均值可以忽略，$E[\Delta X^4] = 3s^4 = 3\sigma^4\Delta t^2$。

两者在 $\lambda = \sqrt{3}$ 时一致。

二叉树。 取 $\lambda = 1$，一步是 $\pm\sigma\sqrt{\Delta t}$，所以 ΔX^4 总是 $\sigma^4\Delta t^2$：系数为 1，而不是 3。两点的一步根本没有尾部。（经过许多步后，由中心极限定理，和仍然变成正态的，所以树照样收敛。）

练习 23.4

思路。 把 $p_d \geq 0$ 写成关于 Δt 的不等式。

第 1 步。 $\dfrac{1}{\lambda^2} \geq \dfrac{\nu\Delta t}{\Delta x} = \dfrac{\nu\sqrt{\Delta t}}{\lambda\sigma}$。整理：$\sqrt{\Delta t} \leq \dfrac{\sigma}{\lambda\nu}$，所以

$$\Delta t \leq \frac{\sigma^2}{\lambda^2\nu^2}.$$

取 $\lambda = \sqrt{3}$。 用课程的数字，$\nu = 0.01$，限制为 $0.04/(3 \times 0.0001) = 133.3$ 年：实际上毫无限制。$r = 25\%$ 时，$\nu = 0.23$，$\Delta t \leq 0.252$ 年，这对非常粗的树才有影响。

如果某个概率为负。 这一步就不再是子节点的平均。一个值可能落在其子节点的范围之外，破坏无套利的逻辑。误差可能增长而不是缩小（第 29 课）。

练习 23.5

思路。 一步，而且只有一个分支有支付。

第 1 步：树。 $\Delta x = \sqrt{3} \times 0.2 = 0.34641$。倾斜量 $= 0.01/0.34641$，所以 $p_u = 0.1811$，$p_m = 0.6667$，$p_d = 0.1522$。

第 2 步：股价。 上：141.40。中：100。下：70.72。

第 3 步：收益。 只有最上面的分支有支付：$141.40 - 105 = 36.40$。

第 4 步：价格。 $0.9704 \times 0.1811 \times 36.40 = 6.40$，对照 7.13。

为什么这么差。 单独一步只有三个结果，无法描述收益的形状：折点落在 100 与 141 之间的一个宽阔间隙中。

验证。 平均贴现股价：$0.9704 \times (0.1811 \times 141.40 + 0.6667 \times 100 + 0.1522 \times 70.72) = 99.99$。它接近 100，但不精确，因为我们匹配的是对数的矩，而不是这个。在精细的树上，差距会消失。

练习 23.6

思路。 三叉树每列增加 2 个节点；二叉树增加 1 个。

三叉树。 i 步之后有 $2i+1$ 个节点。总数为 $\sum_{i=0}^{n}(2i+1) = (n+1)^2$。

二叉树。 i 步之后有 $i+1$ 个节点，总数为 $(n+1)(n+2)/2 \approx n^2/2$。

相同的工作量。 $m^2/2 = n^2$ 给出 $m = \sqrt{2}\,n$ 步二叉树对应 n 步三叉树。（如果数运算次数，三叉树每个节点三次乘法对二叉树两次，就更接近 $\sqrt{3}\,n$。）

内存。 你只需要当前一列：三叉树 $2n+1$ 个数，二叉树 $n+1$ 个。

练习 23.7

三叉树定价程序，以及树上的看跌-看涨平价。

```
import numpy as np


def tree(n, kind="call", lam_=lam, american=False):
    dt = T / n
    d_x = lam_ * sigma * math.sqrt(dt)
    tilt = nu * dt / d_x
    p_u, p_m, p_d = 0.5 * (1 / lam_**2 + tilt), 1 - 1 / lam_**2, 0.5 * (1 / lam_**2 - tilt)
    df = math.exp(-r * dt)
    pay_off = (lambda s: np.maximum(s - K, 0)) if kind == "call" else (lambda s: np.maximum(K
- s, 0))
    v = pay_off(S0 * np.exp(np.arange(-n, n + 1) * d_x))
    for i in range(n - 1, -1, -1):
        v = df * (p_u * v[2:] + p_m * v[1:-1] + p_d * v[:-2])
        if american:
            v = np.maximum(v, pay_off(S0 * np.exp(np.arange(-i, i + 1) * d_x)))
    return float(v[0])


target = S0 - K * math.exp(-r * T)
print("      n       call        put    C - P - (S0 - K e^-rT)")
gaps = {}
for n in (10, 50, 100, 500):
    c, p = tree(n), tree(n, "put")
    gaps[n] = c - p - target
    print(f"{n:7d}   {c:8.4f}   {p:8.4f}   {gaps[n]:+.2e}")
```

它的输出。

```
      n       call        put    C - P - (S0 - K e^-rT)
     10     7.2233     9.1205   -5.01e-04
     50     7.0887     8.9856   -1.00e-04
    100     7.1369     9.0337   -5.00e-05
    500     7.1271     9.0239   -1.00e-05
```

读输出。 平价并不精确。在树上，如果贴现股价是精确的鞅，平价就会精确成立。我们匹配的是对数的矩，而 $p_u e^{\Delta x} + p_m + p_d e^{-\Delta x}$ 与 $e^{r\Delta t}$ 略有不同。差距像 $1/n$ 那样缩小，在 $n = 500$ 时约为 1.0e-05。选择概率使之匹配股价本身的均值，就能使平价精确成立。

练习 23.8

不同的伸缩因子，然后是美式看跌期权。

```
bs = o["bs"]["call"]
print("  lambda    worst error, n = 200 to 209")
worst = {}
for name, l_ in (("1.2", 1.2), ("sqrt 2", math.sqrt(2)), ("sqrt 3", lam), ("2", 2.0)):
    worst[name] = max(abs(tree(n, lam_=l_) - bs) for n in range(200, 210))
    print(f"{name:>8s}  {worst[name]:.5f}")
print("      n   American put")
am = {}
for n in (100, 200, 400, 800):
    am[n] = tree(n, "put", american=True)
    print(f"{n:7d}   {am[n]:.4f}")
```

它的输出。

```
  lambda    worst error, n = 200 to 209
     1.2  0.00111
  sqrt 2  0.00443
  sqrt 3  0.00984
       2  0.00127
       n   American put
     100   9.4784
     200   9.4681
     400   9.4754
     800   9.4751
```

读输出。 最坏误差并不随 λ 单调变化。最要紧的是在这十个大小上行权价落在节点之间的什么位置。$\lambda = \sqrt{3}$ 在这里看起来最差，因为 $n = 200$ 碰巧几乎把行权价放在一个节点上，那里的误差平稳且最大。它的优点在于理论（矩）和实践（稳定的概率），而不在于每次测试中原始误差都更小。$n = 800$ 时的美式看跌期权为 9.475，接近第 22 课的 9.478。

自测

1. $\frac{1}{6}, \frac{2}{3}, \frac{1}{6}$。

2. $\lambda = 1$：这时 $p_m = 0$。

3. 因为误差变得平稳（与 $1/n$ 成正比），而平稳的误差可以通过外推消除。

第 24 课解答

实践中的 delta 对冲

练习 24.1 至 24.8 以及第 24 课自测题的解答。

练习 24.1

思路。 把对冲的收益与期权的泰勒展开比较，然后用方程消去 theta。

第 1 步：期权。 $\delta V \approx \Theta\,\delta t + \Delta\,\delta S + \tfrac{1}{2}\Gamma(\delta S)^2$。

第 2 步：对冲。 Δ 股赚 $\Delta\,\delta S$。现金 $V - \Delta S$ 赚 $r(V - \Delta S)\,\delta t$。

第 3 步：卖方的结果。 对冲的收益减去期权的变化。$\Delta\,\delta S$ 相消：

$$\text{误差} \approx r(V - \Delta S)\,\delta t - \Theta\,\delta t - \tfrac{1}{2}\Gamma(\delta S)^2.$$

第 4 步：方程。 由第 19 课，$rV - rS\Delta - \Theta = \tfrac{1}{2}\sigma^2 S^2\Gamma$。所以

$$\text{误差} \approx \tfrac{1}{2}\sigma^2 S^2\Gamma\,\delta t - \tfrac{1}{2}\Gamma(\delta S)^2 = \tfrac{1}{2}\Gamma S^2\left(\sigma^2\delta t - \left(\tfrac{\delta S}{S}\right)^2\right).$$

练习 24.2

思路。 独立误差的方差相加。

第 1 步：一步。 取 $G = \tfrac{1}{2}\Gamma S^2$ 大致不变，方差为 $G^2\mathrm{Var}((\delta S/S)^2) \approx 2G^2\sigma^4\delta t^2$。

第 2 步：n 步。 取 $\delta t = T/n$，独立误差给出

$$\mathrm{Var}(\text{总误差}) \approx n \times 2G^2\sigma^4\frac{T^2}{n^2} = \frac{2G^2\sigma^4T^2}{n}.$$

第 3 步。 所以标准差与 $1/\sqrt{n}$ 成正比。

比值。 在 $n = 252$ 与 $n = 1000$ 之间：$\sqrt{1000/252} = 1.99$。模拟给出 1.99。

练习 24.3

思路。 当平方移动等于定价时计入的量时，括号为零。

$(\delta S/S)^2 = \sigma^2 \delta t$，所以持平变动幅度是 $\sigma\sqrt{\delta t}$。

- **一周：** $0.2 \times \sqrt{1/52} = 2.77\%$，对价格为 100 的股票即 2.77。
- **一天：** $0.2 \times \sqrt{1/252} = 1.26\%$，即 1.26。

为什么与期权无关。 Gamma 乘在括号外面，所以它决定输赢的多少，但不决定括号在哪里变号。那只取决于波动率和时间步长。

练习 24.4

思路。 在一步误差中，用 $(\delta S/S)^2$ 的真实期望值代替它。

第 1 步：一步。 $(\delta S/S)^2$ 的真实平均值是 $\sigma^2_{\text{true}}\delta t$。所以时刻 t 的期望误差是

$$\tfrac{1}{2}\Gamma_t S_t^2(\sigma^2 - \sigma^2_{\text{true}})\,\delta t.$$

第 2 步：现值。 以 e^{-rt} 贴现并对时间求和：

$$\text{现值} = \tfrac{1}{2}(\sigma^2 - \sigma^2_{\text{true}}) \int_0^T e^{-rt} E[S_t^2 \Gamma_t]\, dt.$$

方向。 $\Gamma > 0$，所以积分为正。当 $\sigma > \sigma_{\text{true}}$ 时，你收费过高，于是获利。当 $\sigma < \sigma_{\text{true}}$ 时，你收费过低，于是亏损。利润只在股价靠近行权价、gamma 较大时累积。

练习 24.5

思路。 一步误差取决于移动的大小，几乎不取决于它的平均值。

漂移给 $\delta S/S$ 加上 $\mu\,\delta t$，所以它的平方多出一项 $2(\mu\,\delta t)(\sigma\sqrt{\delta t}Z) + (\mu\,\delta t)^2$。第一部分均值为 0，第二部分的量级是 δt^2。与 $\sigma^2\delta t$ 相比，两者在首阶上都无关紧要。

μ 确实起作用的地方。 通过股价去向何处：练习 24.4 中的 $E[S_t^2\Gamma_t]$ 依赖于 μ。强烈的漂移把股价带离行权价，所以对冲者持有的 gamma 更少，这会稍微改变分散程度。

为什么这不能帮你挑选股票。 对冲有意地消除了对方向的暴露。如果你对 μ 有看法，最好是持有股票或不加对冲地持有期权。

练习 24.6

思路。 在 σ 上做一阶泰勒展开。

$\mathcal{V} \times (\sigma - \sigma_{\text{true}}) = 39.89 \times (0.20 - 0.25) = -1.994$。

精确的差。 20% 时看涨期权为 7.1281，25% 时为 9.1218，差为 -1.994。非常接近，因为对接近平值的期权，vega 在这个范围内几乎不变。

练习 24.7

每条路径的最终误差与其实现波动率的关系。

```
import numpy as np

Nv = np.vectorize(Nf)


def delta(s, tau, sig=sigma):
    return Nv((np.log(s / K) + (r + 0.5 * sig**2) * tau) / (sig * np.sqrt(tau)))


rng = np.random.default_rng(247)
n, paths = 252, 20_000
dt = T / n
s = np.full(paths, S0)
pos = delta(s, T)
cash = bs(sigma) - pos * s
sum_sq = np.zeros(paths)
for i in range(1, n + 1):
    ret = (mu - 0.5 * sigma**2) * dt + sigma * math.sqrt(dt) * rng.standard_normal(paths)
    s = s * np.exp(ret)
    sum_sq += ret**2
    cash *= math.exp(r * dt)
    if i < n:
        new = delta(s, T - i * dt)
        cash -= (new - pos) * s
        pos = new
err = pos * s + cash - np.maximum(s - K, 0)
real_vol = np.sqrt(sum_sq / T)

print("  realised vol        paths   mean error")
edges = [0.0, 0.19, 0.195, 0.20, 0.205, 0.21, 1.0]
for a, b in zip(edges[:-1], edges[1:]):
    m = (real_vol >= a) & (real_vol < b)
    print(f"  {a:5.3f} to {b:5.3f}   {m.sum():6d}   {err[m].mean():+10.3f}")
corr = np.corrcoef(real_vol, err)[0, 1]
print(f"correlation, all paths: {corr:.3f}")
near, far = np.abs(s - K) < 10, np.abs(s - K) > 40
c_near = np.corrcoef(real_vol[near], err[near])[0, 1]
c_far = np.corrcoef(real_vol[far], err[far])[0, 1]
print(f"finishing within 10 of the strike: {c_near:.3f}; more than 40 away: {c_far:.3f}")
```

它的输出。

```
  realised vol        paths   mean error
  0.000 to 0.190     2752       +0.586
  0.190 to 0.195     3199       +0.297
  0.195 to 0.200     4328       +0.094
  0.200 to 0.205     4213       -0.115
  0.205 to 0.210     3065       -0.309
  0.210 to 1.000     2443       -0.623
correlation, all paths: -0.806
finishing within 10 of the strike: -0.822; more than 40 away: -0.763
```

读输出。 实现波动率低于 20% 的路径赚钱，高于 20% 的亏钱，相关性很强，为 -0.81。这就是一步公式的累加：误差是 $\sigma^2\delta t - (\delta S/S)^2$ 以 gamma 加权的和。在终点靠近行权价的路径中（-0.82），这种联系比终点远离行权价的路径（-0.76）稍紧一些，因为接近平值的路径在后期（也就是要紧的时候）持有更多的 gamma。但这种联系并不完美。波动率**何时**实现，与实现了多少同样重要，因为权重是那一刻的 gamma。

练习 24.8

在同样的路径上，对冲的看跌期权与对冲的看涨期权对比。

```
def hedge_err(n_reb, kind, paths_=20_000, seed=248):
    gen = np.random.default_rng(seed)
    h = T / n_reb
    x = np.full(paths_, S0)
    shift = 0.0 if kind == "call" else -1.0                # the put's delta is the call's
minus 1
    price = bs(sigma) if kind == "call" else bs(sigma) - S0 + K * math.exp(-r * T)
    q = delta(x, T) + shift
    c = price - q * x
    for i in range(1, n_reb + 1):
        x = x * np.exp((mu - 0.5 * sigma**2) * h + sigma * math.sqrt(h) *
gen.standard_normal(paths_))
        c *= math.exp(r * h)
        if i < n_reb:
            new_q = delta(x, T - i * h) + shift
            c -= (new_q - q) * x
            q = new_q
    pay_off = np.maximum(x - K, 0) if kind == "call" else np.maximum(K - x, 0)
    return q * x + c - pay_off


print("      n   sd, put   sd, call   mean, put   largest path gap")
sds = {}
for n_reb in (12, 52, 252):
    ep, ec = hedge_err(n_reb, "put"), hedge_err(n_reb, "call")
    sds[n_reb] = (ep.std(), ec.std(), np.abs(ep - ec).max())
    print(f"{n_reb:7d}   {ep.std():7.4f}   {ec.std():8.4f}   {ep.mean():+9.4f}   {sds[n_reb]
[2]:.1e}")
```

它的输出。

```
      n   sd, put   sd, call   mean, put   largest path gap
     12    2.0412     2.0412     -0.0244   2.0e-13
     52    0.9982     0.9982     -0.0039   6.7e-13
    252    0.4594     0.4594     +0.0045   2.1e-12
```

读输出。 两个标准差不只是接近：它们逐条路径地相同，差别约为 10^{-12}。看跌期权的对冲在每一步都比看涨期权的对冲少持有一股。由平价，$P = C - S + Ke^{-r\tau}$：看跌期权是看涨期权加上股票和现金的一个静态头寸。静态头寸不需要再平衡，也没有对冲误差，所以两个误差完全相同。

自测

1. 它减半。

2. 1.26%。

3. 你会亏损，到 T 时约 2.1：大致是 25% 与 20% 下 Black-Scholes 价格之差，换算到到期时。

第 25 课解答

Girsanov 定理

练习 25.1 至 25.8 以及第 25 课自测题的解答。

练习 25.1

思路。 W_T 是正态的，所以它的指数有已知的平均值。

$E[Z_T] = 1$。 $W_T \sim N(0, T)$，所以由矩母函数（取 $u = -\theta$）：$E[e^{-\theta W_T}] = e^{\frac{1}{2}\theta^2 T}$。因此

$$E[Z_T] = e^{-\frac{1}{2}\theta^2 T} e^{\frac{1}{2}\theta^2 T} = 1.$$

鞅。 拆分 $W_T = W_t + D$，其中 $D = W_T - W_t$ 与 $\mathcal{F}_t$ 独立，服从 $N(0, T-t)$。

$$E[Z_T \mid \mathcal{F}_t] = e^{-\theta W_t - \frac{1}{2}\theta^2 T} E[e^{-\theta D}] = e^{-\theta W_t - \frac{1}{2}\theta^2 T} e^{\frac{1}{2}\theta^2 (T-t)} = e^{-\theta W_t - \frac{1}{2}\theta^2 t} = Z_t.$$

练习 25.2

思路。 $Z_t = f(t, W_t)$，其中 $f(t, w) = e^{-\theta w - \frac{1}{2}\theta^2 t}$。

第 1 步：导数。 $f_t = -\frac{1}{2}\theta^2 f$，$f_w = -\theta f$，$f_{ww} = \theta^2 f$。

第 2 步：伊藤引理。

$$dZ = \left(-\tfrac{1}{2}\theta^2 Z + \tfrac{1}{2}\theta^2 Z\right) dt - \theta Z \, dW = -\theta Z \, dW.$$

为什么这意味着鞅。 在可积的条件下（这里成立），伊藤积分 $\int H \, dW$ 在任何未来区间上的条件均值为零（第 15 课）。没有 dt 项，就没有东西推动平均值。

数值。 在 $W_T = 1$ 处：$e^{-0.25 - 0.03125} = 0.7548$。在 $W_T = -1$ 处：$e^{0.25 - 0.03125} = 1.2445$。

练习 25.3

思路。 每条公理都由 Z_T 的一个性质推出。

- **非负：** $Z_T > 0$ 且 $\mathbf{1}_A \geq 0$，所以 $E[Z_T \mathbf{1}_A] \geq 0$。
- **总和为 1：** $\tilde{P}(\Omega) = E[Z_T] = 1$。
- **可加：** 对不交事件，$\mathbf{1}_{A \cup B} = \mathbf{1}_A + \mathbf{1}_B$，而平均是可加的。（对可数并也一样，由单调收敛。）

等价。 如果 $P(A) = 0$，那么以概率 1 有 $Z_T \mathbf{1}_A = 0$，所以 $\tilde{P}(A) = 0$。如果 $\tilde{P}(A) = 0$，那么 $Z_T \mathbf{1}_A$ 是非负的且平均为零，所以它以概率 1 为零。由于 $Z_T > 0$，以概率 1 有 $\mathbf{1}_A = 0$，所以 $P(A) = 0$。

如果 Z_T 可能为 0。 一个在 P 下可能发生的事件，在 $\tilde{P}$ 下可能变得不可能。反方向的论证失效，两者就不再等价。在 $\tilde{P}$ 下，对冲可能遗漏一些真实可能发生的情景。

练习 25.4

思路。 合并指数，矩母函数完成其余的工作。

第 1 步。 $Z_T e^{u(W_T + \theta T)} = \exp\left((u - \theta) W_T + u\theta T - \frac{1}{2}\theta^2 T\right)$。

第 2 步。 对 $W_T \sim N(0, T)$ 取平均：$E[e^{(u-\theta) W_T}] = e^{\frac{1}{2}(u-\theta)^2 T}$。

第 3 步。 总指数：

$$\tfrac{1}{2}(u^2 - 2u\theta + \theta^2)T + u\theta T - \tfrac{1}{2}\theta^2 T = \tfrac{1}{2}u^2 T.$$

所以 $\tilde{E}[e^{u\tilde{W}_T}] = e^{\frac{1}{2}u^2 T}$，即 $N(0, T)$ 的矩母函数。矩母函数决定分布，所以在 $\tilde{P}$ 下 $\tilde{W}_T \sim N(0, T)$。

练习 25.5

思路。 代入 $dW = d\tilde{W} - \theta\, dt$。

第 1 步。 $dS = \mu S\, dt + \sigma S(d\tilde{W} - \theta\, dt) = (\mu - \sigma\theta) S\, dt + \sigma S\, d\tilde{W} = r S\, dt + \sigma S\, d\tilde{W}$。

第 2 步：求解， 同第 17 课：$S_T = S_0 e^{(r - \frac{1}{2}\sigma^2)T + \sigma \tilde{W}_T}$。

第 3 步：在 $\tilde{P}$ 下。 $\tilde{W}_T \sim N(0, T)$，所以

$$\tilde{P}(S_T > K) = N\Big(\frac{\ln(S_0/K) + (r - \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}\Big) = N(-0.1940) = 0.4231.$$

第 4 步：在 P 下。 $S_T = S_0 e^{(\mu - \frac{1}{2}\sigma^2)T + \sigma W_T}$，所以用 μ 代入同样的公式得到 $N(0.0560) = 0.5223$。

现实世界的概率高出 0.0992：同一个事件，不同的权重。

练习 25.6

思路。 Z_T^2 仍然是正态变量的指数。

第 1 步。 $Z_T^2 = e^{-2\theta W_T - \theta^2 T}$。由 $E[e^{-2\theta W_T}] = e^{2\theta^2 T}$，得 $E[Z_T^2] = e^{\theta^2 T}$。

第 2 步：数字。

- $\theta = 0.25$：$E[Z_T^2] = 1.0645$，有效路径比例为 0.939。
- $\theta = 3$：$E[Z_T^2] =$ 8103，有效比例为 1.2e-04。一百万条路径中，约有 123 条有效。

为什么它危险。 θ 很大时，几乎全部权重都集中在尾部远处的少数几条路径上。估计值依赖于这几条路径，而它们甚至可能根本没被抽到。更糟的是，错过了它们的一次运行看起来很平静，它自己的误差棒可能错得离谱。

练习 25.7

看涨和看跌期权，加权与不加权。

```
import numpy as np

rng = np.random.default_rng(257)
M = 1_000_000
Z = rng.standard_normal(M)
disc = math.exp(-r * T)
pays = {"call": lambda s: disc * np.maximum(s - K, 0), "put": lambda s: disc * np.maximum(K -
s, 0)}
print(f"formula: call {o['bs']['call']:.4f}, put {o['bs']['put']:.4f}")
print("                                    call (se)            put (se)")
S_rn = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)
a, b = pays["call"](S_rn), pays["put"](S_rn)
print(f"directly under the new probability  {a.mean():7.4f} ({a.std() / 1000:.4f})
{b.mean():7.4f} ({b.std() / 1000:.4f})")
se = {}
for mu in (0.08, 0.50):
    theta = (mu - r) / sigma
    W = math.sqrt(T) * Z                                    # W is Brownian under P
    S_T = S0 * np.exp((mu - 0.5 * sigma**2) * T + sigma * W)
    weight = np.exp(-theta * W - 0.5 * theta**2 * T)
    a, b = pays["call"](S_T) * weight, pays["put"](S_T) * weight
    se[mu] = (a.std() / 1000, b.std() / 1000)
    print(f"under P, mu = {mu:3.0%}, weighted            {a.mean():7.4f} ({se[mu][0]:.4f})
{b.mean():7.4f} ({se[mu][1]:.4f})")
```

它的输出。

```
formula: call 7.1281, put 9.0248
                                    call (se)            put (se)
directly under the new probability   7.1551 (0.0126)    9.0174 (0.0111)
under P, mu =  8%, weighted             7.1474 (0.0094)    9.0160 (0.0149)
under P, mu = 50%, weighted             7.1296 (0.0088)    8.6541 (0.3488)
```

读输出。 这道题值得仔细看，结果并不是上一道练习可能让你预期的那样。

看涨期权的误差随 μ 升高而缩小： 8% 时为 0.0094，50% 时为 0.0088。向上漂移的路径更常落在看涨期权有支付的地方。那里的权重很小，这使乘积保持在可控范围内。这就是用得好的**重要性抽样**：多抽取要紧的部分，再用权重修正。

看跌期权的误差爆炸了： 8% 时为 0.0149，50% 时为 0.3488。现在路径很少落在看跌期权有支付的地方，而一旦落在那里，权重就极大。估计值离 9.02 很远，它报告的误差本身也不可靠，因为它取决于起主导作用的那几条路径。

教训。 概率变换本身无所谓好坏。它的效果取决于它是把样本推向收益，还是推离收益。

练习 25.8

二次变差，加权与不加权。

```
rng = np.random.default_rng(258)
paths, n = 2000, 2000
dW = rng.normal(0.0, math.sqrt(T / n), (paths, n))
Z_T = np.exp(-th * dW.sum(axis=1) - 0.5 * th**2 * T)
w = Z_T / Z_T.sum()
qv = (dW**2).sum(axis=1)


def weighted_quantile(x, weights, q):
    order = np.argsort(x)
    return float(x[order][np.searchsorted(weights[order].cumsum(), q)])


print("            plain     weighted")
print(f"mean QV    {qv.mean():8.4f}  {(w * qv).sum():8.4f}")
for q in (0.05, 0.5, 0.95):
    print(f"{q:4.0%} QV   {np.quantile(qv, q):8.4f}  {weighted_quantile(qv, w, q):8.4f}")

half = dW[:, : n // 2].sum(axis=1)
Wt_half, Wt_end = half + th * 0.5, dW.sum(axis=1) + th * T
print(f"weighted mean of (W tilde)^2 - t: t = 0.5: {(w * (Wt_half**2 - 0.5)).sum():+.4f}, t =
1: {(w * (Wt_end**2 - 1)).sum():+.4f}")
```

它的输出。

```
            plain     weighted
mean QV      0.9993    0.9997
  5% QV     0.9488    0.9492
 50% QV     0.9988    0.9990
 95% QV     1.0540    1.0546
weighted mean of (W tilde)^2 - t: t = 0.5: -0.0205, t = 1: -0.0134
```

读输出。 实现的 $\sum(\Delta W)^2$ 的均值和三个分位数几乎不随权重变化，它们也不可能变化：二次变差是每条路径的性质，在所有路径上几乎是同一个数。鞅 $\tilde{W}_t^2 - t$ 的加权均值在抽样误差范围内为零，正如在 $\tilde{P}$ 下的布朗运动应有的那样。（只有 2000 条路径时，这两个均值的噪声约为 0.03。）

自测

1. -0.25。

2. $\tilde{W}_t = W_t + \theta t$。

3. 因为二次变差是每条路径的性质，以概率 1 成立。概率变换给路径重新加权，而不改变它们。等价的概率有相同的概率为 1 的事件。

第 26 课解答

风险中性定价

练习 26.1 至 26.8 以及第 26 课自测题的解答。

练习 26.1

思路。 乘积法则，而贴现因子没有随机性。

第 1 步。 $D_t = e^{-rt}$ 满足 $dD = -rD\,dt$，没有 dW 部分。

第 2 步。 所以由乘法表 $dt\,dt = dt\,dW = 0$，$dD\,dS = -rD\,dt\,(\mu S\,dt + \sigma S\,dW) = 0$。**这就是为什么没有交叉项。**

第 3 步。

$$d(DS) = D\,dS + S\,dD = D(\mu S\,dt + \sigma S\,dW) - rDS\,dt = (\mu - r)DS\,dt + \sigma DS\,dW.$$

第 4 步。 提出 σDS：$\sigma DS\left(\frac{\mu - r}{\sigma}\,dt + dW\right) = \sigma DS(\theta\,dt + dW)$。

练习 26.2

思路。 θ 度量单位风险的回报。

数值。 $\theta = (0.08 - 0.03)/0.2 = 0.25$。

$\theta = 0$。 它意味着 $\mu = r$：风险没有回报。这时 $Z_T = 1$，所以 $\tilde{P} = P$。现实世界本身就是风险中性的。

平均价格。 在 $\tilde{P}$ 下，漂移为 r：$\tilde{E}[S_T] = S_0 e^{rT} = 103.05$。在 P 下，漂移为 μ：$E[S_T] = S_0 e^{\mu T} = 108.33$。

练习 26.3

思路。 贴现去掉了现金，只剩下贴现股票。

第 1 步。 由练习 19.1：$d(DX) = \Delta\,d(DS)$。

第 2 步。 所以 $d(DX) = \Delta\sigma DS\, d\tilde{W}$，没有 dt 项。对任何合理的 Δ，这是一个 $\tilde{P}$-鞅。

不可能稳赚。 设 $X_0 = 0$，$X_T \geq 0$ 以概率 1 成立，且 $P(X_T > 0) > 0$。由于两个概率等价，在 $\tilde{P}$ 下同样成立。所以 $\tilde{E}[D_T X_T] > 0$。但鞅性质说 $\tilde{E}[D_T X_T] = X_0 = 0$。矛盾。

注意用到了什么。 Δ 多么巧妙都无关紧要：只要 Δ 只用到过去，这个论证就对所有 Δ 适用。

练习 26.4

思路。 贴现股价是鞅，所以它的平均值是 S_0。

$$V_0 = \tilde{E}[e^{-rT}(S_T - F)] = \tilde{E}[e^{-rT}S_T] - e^{-rT}F = S_0 - e^{-rT}F.$$

为零，当 $F = S_0 e^{rT} = 103.0455$。

不需要什么。 既不需要 σ，也不需要 μ。远期合约可以静态对冲：买入股票，借入它的价款。它不依赖于任何关于股价如何变动的模型。

练习 26.5

思路。 对 $\tilde{W}^2$ 用伊藤引理，同第 15 课。

第 1 步。 $d(\tilde{W}_t^2) = 2\tilde{W}_t\, d\tilde{W}_t + dt$。从 0 到 T 积分：$\tilde{W}_T^2 - T = \int_0^T 2\tilde{W}_t\, d\tilde{W}_t$。

第 2 步。 所以 $\Gamma_t = 2\tilde{W}_t$。

对冲。 取 $r = 0$，$D = 1$，$\Delta_t = \Gamma_t/(\sigma S_t) = \dfrac{2\tilde{W}_t}{\sigma S_t}$ 股。当 $\tilde{W}_t$ 的绝对值很大时，这个权益的价值对进一步的变动很敏感，所以需要更多的股票。

练习 26.6

思路。 选择权重以消去波动率。无风险的组合必须赚取 r。

第 1 步：权重。 把财富的 w_1 投入资产 1，$w_2 = 1 - w_1$ 投入资产 2。它的收益率是 $(w_1\mu_1 + w_2\mu_2)\, dt + (w_1\sigma_1 + w_2\sigma_2)\, dW$。

第 2 步：消去 dW。 $w_1\sigma_1 + (1 - w_1)\sigma_2 = 0$，所以 $w_1 = \sigma_2/(\sigma_2 - \sigma_1) = 0.3/0.1 = 3$，$w_2 = $ -2。

第 3 步：它的收益率。 $3 \times 8\% - 2 \times 14\% = -0.04$，即每年 -4%，完全没有风险。与 3% 的现金比较。

套利。 卖空这个组合（即每单位持有 -3 份资产 1 和 $+2$ 份资产 2），把所得投入现金。你每年赚 $3\% - (-4\%) = 7\%$，没有风险，也不用出钱。

哪个 μ_2 能消除它？ 两个风险市场价格必须相等：$(\mu_2 - r)/\sigma_2 = (\mu_1 - r)/\sigma_1 = 0.25$。所以 $\mu_2 = 3\% + 0.25 \times 30\% = 0.105$，即 10.5%。

练习 26.7

两种收益，各用两种方式模拟。

```
import numpy as np

theta = (mu - r) / sigma
dm = (math.log(S0 / K) + (r - 0.5 * sigma**2) * T) / sigma
dp = dm + sigma
disc = math.exp(-r * T)
exact = {"call": S0 * Nf(dp) - K * disc * Nf(dm), "digital": disc * Nf(dm)}

rng = np.random.default_rng(267)
M = 1_000_000
Z = rng.standard_normal(M)
S_rn = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * Z)
S_re = S0 * np.exp((mu - 0.5 * sigma**2) * T + sigma * Z)
wt = np.exp(-theta * Z - 0.5 * theta**2 * T)
pays = {"call": lambda s: disc * np.maximum(s - K, 0), "digital": lambda s: disc * (s > K)}
print("          exact     risk-neutral (se)     real with weights (se)   se ratio")
ratios = {}
for name, f in pays.items():
    a, b = f(S_rn), f(S_re) * wt
    ratios[name] = (b.std() / a.std())
    print(f"{name:8s}  {exact[name]:7.4f}  {a.mean():7.4f} ({a.std() / 1000:.5f})
{b.mean():7.4f} ({b.std() / 1000:.5f})      {ratios[name]:.2f}")
```

它的输出。

```
          exact     risk-neutral (se)     real with weights (se)   se ratio
call       7.1281   7.1100 (0.01253)     7.1112 (0.00937)      0.75
digital    0.4106   0.4095 (0.00048)     0.4100 (0.00040)      0.84
```

读输出。 在这里，权重没有坏处：加权后的标准误差更低，看涨期权降低为 0.75 倍，数字期权为 0.84 倍。两种收益都集中在高股价处。以 $\mu = 8\%$ 模拟让更多路径落在那里，而那里的权重小于 1。看涨期权获益更多，因为它在最右侧的尾部有更多分量。这就是重要性抽样，同练习 25.7。

练习 26.8

由鞅的积分重建看涨期权的贴现收益。

```
Nv = np.vectorize(Nf)


def bs_call(s, tau):
    d1 = (np.log(s / K) + (r + 0.5 * sigma**2) * tau) / (sigma * np.sqrt(tau))
    return s * Nv(d1) - K * np.exp(-r * tau) * Nv(d1 - sigma * np.sqrt(tau))


rng = np.random.default_rng(268)
paths = 200
fine = 2000
dW = rng.normal(0.0, math.sqrt(T / fine), (paths, fine))
print("  steps   rms gap")
rms = {}
for n in (500, 1000, 2000):
    h = T / n
    inc = dW.reshape(paths, n, fine // n).sum(axis=2)         # the same paths at n coarser
steps
    s = np.full(paths, S0)
    M_ = np.full(paths, float(bs_call(S0, T)))
    for i in range(n):
        t = i * h
        d1 = (np.log(s / K) + (r + 0.5 * sigma**2) * (T - t)) / (sigma * math.sqrt(T - t))
        M_ += sigma * math.exp(-r * t) * s * Nv(d1) * inc[:, i]
        s = s * np.exp((r - 0.5 * sigma**2) * h + sigma * inc[:, i])
    gap = M_ - math.exp(-r * T) * np.maximum(s - K, 0)
    rms[n] = float(np.sqrt((gap**2).mean()))
    print(f"{n:7d}   {rms[n]:.4f}")
```

它的输出。

```
  steps   rms gap
    500   0.4147
   1000   0.2892
   2000   0.2061
```

读输出。 步数每翻一倍，均方根差距大约缩小 $\sqrt{2}$ 倍，所以像 $1/\sqrt{n}$ 那样下降。这与第 24 课对冲误差的速率相同，原因也相同：$M_0 + \sum \Gamma \, \Delta \tilde{W}$ 就是一个离散再平衡的 delta 对冲的贴现财富。从 500 步到 2000 步，误差缩小 2.01 倍，接近 $\sqrt{4} = 2$。

自测

1. $\theta = 0.25$：股票单位波动率的超额收益，即它的 Sharpe 比率。

2. 零。

3. 对冲的存在性及其形式。定价公式说的是可复制的收益必须值多少。表示定理说的是每个收益都可以被复制，以及怎样复制。

第 27 课解答

计价单位变换

练习 27.1 至 27.8 以及第 27 课自测题的解答。

练习 27.1

思路。 两个不同的论证，都很短。

用鞅性质。 在 $\tilde{P}$ 下，$e^{-rt}S_t$ 是从 S_0 出发的鞅。所以 $\tilde{E}[e^{-rT}S_T] = S_0$，除以 S_0 得 1。

用正态分布。 $e^{-rT}S_T/S_0 = e^{-\frac{1}{2}\sigma^2 T + \sigma\tilde{W}_T}$。由 $\tilde{W}_T \sim N(0,T)$，矩母函数给出 $E[e^{\sigma\tilde{W}_T}] = e^{\frac{1}{2}\sigma^2 T}$。乘积为 1。

练习 27.2

思路。 用权重计算 $\ln S_T$ 在 P^S 下的矩母函数。

第 1 步。 令 $Y = \ln(S_T/S_0) = (r - \frac{1}{2}\sigma^2)T + \sigma\tilde{W}_T$。权重为 $Z_T^S = e^{\sigma\tilde{W}_T - \frac{1}{2}\sigma^2 T}$。

第 2 步。

$$E^S[e^{uY}] = \tilde{E}[Z_T^S e^{uY}] = e^{u(r-\frac{1}{2}\sigma^2)T - \frac{1}{2}\sigma^2 T}\tilde{E}[e^{(u+1)\sigma\tilde{W}_T}].$$

第 3 步。 矩母函数给出 $\tilde{E}[e^{(u+1)\sigma\tilde{W}_T}] = e^{\frac{1}{2}(u+1)^2\sigma^2 T}$。合并指数：

$$u(r-\tfrac{1}{2}\sigma^2)T - \tfrac{1}{2}\sigma^2 T + \tfrac{1}{2}(u^2+2u+1)\sigma^2 T = u(r+\tfrac{1}{2}\sigma^2)T + \tfrac{1}{2}u^2\sigma^2 T.$$

这是 $N\big((r+\frac{1}{2}\sigma^2)T,\ \sigma^2 T\big)$ 的矩母函数。 所以 $\ln S_T$ 是均值为 $\ln S_0 + (r+\frac{1}{2}\sigma^2)T$、方差为 $\sigma^2 T$ 的正态变量。

S_T 的**中位数**是 $S_0 e^{(r+\frac{1}{2}\sigma^2)T} = 100e^{0.05} = 105.13$。

练习 27.3

思路。 写 $Y = f(t,S)$，让伊藤引理表明漂移为零。

第 1 步。 $f(t,S)=e^{rt}/S$：$f_t=rf$，$f_S=-f/S$，$f_{SS}=2f/S^2$。

第 2 步：伊藤引理，取 $a=(r+\sigma^2)S$，$b=\sigma S$：

$$dY=\Big(rY-(r+\sigma^2)Y+\tfrac12\sigma^2S^2\frac{2Y}{S^2}\Big)\,dt-\sigma Y\,dW^S.$$

第 3 步。 dt 括号为 $rY-rY-\sigma^2Y+\sigma^2Y=0$。所以 $dY=-\sigma Y\,dW^S$：在 P^S 下是鞅。

练习 27.4

思路。 在 P^S 下漂移不同，但标准化的计算相同。

第 1 步。 在 P^S 下，$\ln S_T=\ln S_0+(r+\tfrac12\sigma^2)T+\sigma\sqrt{T}Z$，$Z\sim N(0,1)$。

第 2 步。 $S_T>K\iff Z>-\dfrac{\ln(S_0/K)+(r+\frac12\sigma^2)T}{\sigma\sqrt{T}}=-d_+$。所以 $P^S(S_T>K)=N(d_+)$。

两项。

- 行权价项 $K\mathbf{1}\{S_T>K\}$ 是一笔固定的现金，所以用现金计价单位：$Ke^{-rT}\tilde P(S_T>K)=Ke^{-rT}N(d_-)$。
- 股票项 $S_T\mathbf{1}\{S_T>K\}$ 是固定数量的股票，所以用股票计价单位：$S_0E^S[\mathbf{1}\{S_T>K\}]=S_0N(d_+)$。

为什么 $N(d_+)>N(d_-)$。 P^S 给每条路径的权重与 S_T 成正比，所以它偏爱的恰恰是股价终点高的路径。这些正是实值的路径。所以同一个事件的概率更高。

练习 27.5

思路。 以资产 2 为单位，这是一个行权价为 1、利率为零的普通看涨期权。

第 1 步：计价单位。 使用 P^2，权重为 $Z=e^{-rT}S_T^2/S_0^2$。由第 27 课的论证，

$$V_0=S_0^2E^2\Big[\frac{\max(S_T^1-S_T^2,0)}{S_T^2}\Big]=S_0^2E^2[\max(Y_T-1,0)],\qquad Y=S^1/S^2.$$

第 2 步：Y 的运动。 Y 是以计价单位计的可交易资产，所以是 P^2-鞅：没有漂移。

第 3 步：Y 的波动率。 用风险中性形式 $dS^i=rS^i\,dt+\sigma_iS^i\,d\tilde W^i$，$d\ln Y=d\ln S^1-d\ln S^2$ 的随机部分是 $\sigma_1\,d\tilde W^1-\sigma_2\,d\tilde W^2$。它每单位时间的二次变差为

$$\sigma_x^2 = \sigma_1^2 - 2\rho\sigma_1\sigma_2 + \sigma_2^2.$$

概率变换不影响这一点（第 25 课）。

第 4 步：读出结果。 Y 是对数正态的，波动率为 σ_x，没有漂移，起点为 $Y_0 = S_0^1/S_0^2$。所以 $E^2[\max(Y_T - 1, 0)]$ 是行权价为 1、$r = 0$ 的 Black-Scholes 看涨期权：$Y_0 N(d_1) - N(d_2)$。乘以 S_0^2 得到 $V_0 = S_0^1 N(d_1) - S_0^2 N(d_2)$。

数字。 $\sigma_x^2 = 0.04 + 0.09 - 2 \times 0.5 \times 0.06 = 0.07$，所以 $\sigma_x = 0.2646$。而 $d_1 = 0.3262$，$d_2 = 0.0616$，$N(d_1) = 0.6278$，$N(d_2) = 0.5246$。所以

$$V_0 = 100 \times 0.6278 - 95 \times 0.5246 = 12.9523.$$

练习 27.6

思路。 看跌期权在股价终止于 K 以下时支付，所以两个概率都是这个事件的概率。

拆分。 $P = Ke^{-rT}\tilde{P}(S_T < K) - S_0 P^S(S_T < K)$，与看涨期权同样的论证，第一项用现金，第二项用股票。

概率。 $\tilde{P}(S_T < K) = 1 - N(d_-) = N(-d_-) = 0.5769$，$P^S(S_T < K) = N(-d_+) = 0.4976$。

为什么是“低于”。 看跌期权只在那里支付。股票单位降低了终止于 K 以下的概率，因为它偏爱高的路径：$N(-d_+) < N(-d_-)$。

练习 27.7

N(d₊)，直接估计与加权估计。

```
import numpy as np

M = 1_000_000
print(" strike    N(d+)     under P^S (se)        weighted from Q (se)")
ses = {}
for k in (105.0, 160.0):
    dp = (math.log(S0 / k) + (r + 0.5 * sigma**2) * T) / sigma
    rng = np.random.default_rng(277)
    Z = rng.standard_normal(M)
    direct = (S0 * np.exp((r + 0.5 * sigma**2) * T + sigma * Z) > k).astype(float)
    S_Q = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * Z)
    weighted = math.exp(-r * T) * S_Q / S0 * (S_Q > k)
    ses[k] = (direct.std() / 1000, weighted.std() / 1000)
    print(f"{k:7.0f}  {N(dp):8.5f}  {direct.mean():8.5f} ({ses[k][0]:.5f})
{weighted.mean():8.5f} ({ses[k][1]:.5f})")
```

它的输出。

```
 strike    N(d+)     under P^S (se)        weighted from Q (se)
    105   0.50241   0.50235 (0.00050)     0.50208 (0.00059)
    160   0.01786   0.01781 (0.00013)     0.01784 (0.00017)
```

读输出。 这里直接模拟的标准误差更低，在 $K = 105$ 时低 1.19 倍，在 $K = 160$ 时低 1.30 倍。直接估计量是纯粹的抛硬币，方差为 $p(1-p)$。加权估计量把硬币乘以一个逐路径变化的权重，这增加了方差。在 $K = 160$ 时，权重通过把更多价值放在要紧的路径上而起到一些作用，但不足以胜过从正确分布直接抽样。当你能在正确的概率下直接模拟时，就那样做。

练习 27.8

Margrabe 公式与蒙特卡洛对比。

```
def margrabe(a0, b0, s_a, s_b, rho):
    v = math.sqrt(s_a**2 + s_b**2 - 2 * rho * s_a * s_b)
    d_1 = (math.log(a0 / b0) + 0.5 * v**2 * T) / (v * math.sqrt(T))
    return a0 * N(d_1) - b0 * N(d_1 - v * math.sqrt(T))


a0, b0, s_a, s_b = 100.0, 95.0, 0.2, 0.3
rng = np.random.default_rng(278)
z1, z_ind = rng.standard_normal(M), rng.standard_normal(M)
print("   rho   formula   simulated, r = 0     r = 10%")
vals = {}
for rho in (-0.5, 0.0, 0.5, 0.9):
    z2 = rho * z1 + math.sqrt(1 - rho**2) * z_ind
    res = []
    for rate in (0.0, 0.10):
        A = a0 * np.exp((rate - 0.5 * s_a**2) * T + s_a * z1)
        B = b0 * np.exp((rate - 0.5 * s_b**2) * T + s_b * z2)
        res.append(math.exp(-rate * T) * np.maximum(A - B, 0).mean())
    vals[rho] = margrabe(a0, b0, s_a, s_b, rho)
    print(f"{rho:6.1f}   {vals[rho]:7.4f}   {res[0]:16.4f}   {res[1]:9.4f}")
    assert abs(res[0] - vals[rho]) < 0.08 and abs(res[0] - res[1]) < 1e-9
```

它的输出。

```
   rho   formula   simulated, r = 0     r = 10%
  -0.5   19.4360            19.4522     19.4522
   0.0   16.5882            16.6041     16.6041
   0.5   12.9523            12.9656     12.9656
   0.9    8.6046             8.6143      8.6143
```

读输出。 公式在抽样误差范围内相符。价格随 ρ 升高而**下降**。当两种资产一起变动时，它们的比值几乎不动，所以选择权的价值很小：$\sigma_x^2 = \sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2$ 缩小了。在 $\rho = -0.5$ 时，两种资产背道而驰，交换的权利就值钱得多。

利率。 把 r 从 0 改到 10% 不会改变任何东西：在同样的随机数上，结果完全一致。两种资产都得到因子 e^{rT}，而贴现 e^{-rT} 把它抵消了。

自测

1. $r + \sigma^2$，用课程的数字是 7%。

2. 以股票为单位时，看涨期权到期处于实值的概率：0.5024。（以现金为单位时，是 0.4231。）

3. 两种资产以同样的风险中性速率增长，所以只有它们的比值要紧。以资产 2 为单位计，比值没有漂移，贴现也就消失了。

第 28 课解答

Feynman-Kac 定理

练习 28.1 至 28.8 以及第 28 课自测题的解答。

练习 28.1

思路。 以今天为条件，对预测的预测就是预测。

鞅。 令 $Y_t = E[h(X_T) \mid \mathcal{F}_t]$。对 $s < t$，塔性质给出

$$E[Y_t \mid \mathcal{F}_s] = E\big[E[h(X_T) \mid \mathcal{F}_t] \mid \mathcal{F}_s\big] = E[h(X_T) \mid \mathcal{F}_s] = Y_s.$$

马尔可夫性质。 原则上，Y_t 可能依赖于直到 t 的整条路径。由于 $dX = a(X)\,dt + b(X)\,dW$，未来的变动只依赖于当前值和新的噪声。所以条件期望只通过 X_t 依赖于过去，对某个普通函数 g 有 $Y_t = g(t, X_t)$。没有这一点，就没有关于 (t, x) 的函数可以求导。

练习 28.2

思路。 取 $a = 0$、$b = 1$：$g_t + \tfrac{1}{2} g_{xx} = 0$。

$h = x^2$。猜 $g = x^2 + c(t)$。那么 $g_t = c'$，$g_{xx} = 2$，所以 $c' + 1 = 0$。由 $g(T, x) = x^2$，需要 $c(T) = 0$，所以 $c = T - t$。因此 $g = x^2 + T - t$。

验证。 $E[(x + \sqrt{T-t}\,Z)^2] = x^2 + (T - t)$。$\checkmark$

$h = e^x$。猜 $g = e^{x + c(t)}$。那么 $g_t = c' g$，$g_{xx} = g$，所以 $c' + \tfrac{1}{2} = 0$。由 $c(T) = 0$，$c = \tfrac{1}{2}(T - t)$。因此 $g = e^{x + \frac{1}{2}(T-t)}$。

验证。 由正态分布的矩母函数，$E[e^{x + \sqrt{T-t}\,Z}] = e^x e^{\frac{1}{2}(T-t)}$。$\checkmark$

练习 28.3

思路。 是贴现后的过程为鞅。

第 1 步。 $e^{-rt}g(t,X_t)=e^{-rt}e^{-r(T-t)}E[h(X_T)\mid\mathcal{F}_t]=e^{-rT}E[h(X_T)\mid\mathcal{F}_t]$。这是常数乘以条件期望，由塔性质是鞅。

第 2 步：乘积法则和伊藤引理。

$$d(e^{-rt}g)=e^{-rt}\,dg-re^{-rt}g\,dt=e^{-rt}\big(g_t+ag_x+\tfrac12 b^2g_{xx}-rg\big)\,dt+e^{-rt}bg_x\,dW.$$

第 3 步。 鞅没有 dt 项，而 $e^{-rt}>0$。所以 $g_t+ag_x+\frac12 b^2g_{xx}=rg$。

练习 28.4

思路。 方程关于 S 是线性的，所以解是 S 乘以一个时间的函数。

不贴现。 $g_t+\mu Sg_S+\frac12\sigma^2S^2g_{SS}=0$。取 $g=Sc(t)$：$g_t=Sc'$，$g_S=c$，$g_{SS}=0$。所以 $c'+\mu c=0$，$c(T)=1$。因此 $c=e^{\mu(T-t)}$，$g=Se^{\mu(T-t)}$。用我们的数字，从 $t=0$ 时的 100 出发：108.33。

贴现。 右边变成 rg：$c'+\mu c=rc$。所以 $c=e^{(\mu-r)(T-t)}$，$g=Se^{(\mu-r)(T-t)}$，即 105.13。

当 $\mu=r$ 时： $c=1$，所以 $g=S$。股票的贴现平均值就是今天的股价。一股就值它自己的价格。

练习 28.5

思路。 每个期望都满足它自己的方程。定理并没有说哪一个是价格。

对的部分。 Feynman-Kac 适用于任何漂移 a。取 $a=\mu S$，$g_\mu=E[e^{-r(T-t)}h(S_T)\mid S_t=S]$ 满足 $g_t+\mu Sg_S+\frac12\sigma^2S^2g_{SS}=rg$。

错的部分。 那不是 Black-Scholes 方程，后者在这里是 μ 的地方是 r。第 19 课的对冲论证说，期权的价值必须满足含 r 的方程，因此由取 $a=rS$ 的 Feynman-Kac，它是漂移为 r 的期望。

多余的项。 把 g_μ 代入 Black-Scholes 方程：由它自己的方程，$g_t+\frac12\sigma^2S^2g_{SS}=rg-\mu Sg_S$。所以

$$g_t+rSg_S+\tfrac12\sigma^2S^2g_{SS}-rg=-(\mu-r)Sg_S.$$

它差了 $(\mu-r)Sg_S$，在 $S=100$、$t=0.5$ 处为 2.5808。而在价值上，9.96 并不是期权的价格 7.13。

练习 28.6

思路。 方程必须对每个 r 都成立，所以 r 的系数和其余部分必须分别为零。

第 1 步：导数。 取 $P = e^{A(\tau) - B(\tau) r}$，$\tau = T - t$：$P_t = -(A' - B' r) P$，$P_r = -BP$，$P_{rr} = B^2 P$。

第 2 步：代入，并除以 P。

$$-A' + B' r - \kappa(\bar{r} - r) B + \tfrac{1}{2} s^2 B^2 = r.$$

第 3 步：含 r 的项：$B' + \kappa B = 1$，$B(0) = 0$。所以 $B = (1 - e^{-\kappa \tau})/\kappa$。

第 4 步：不含 r 的项：$A' = -\kappa \bar{r} B + \tfrac{1}{2} s^2 B^2$，$A(0) = 0$。

第 5 步：积分。 利用 $\int_0^\tau B\, du = (\tau - B)/\kappa$ 和 $\int_0^\tau B^2\, du = \dfrac{1}{\kappa^2}(\tau - B) - \dfrac{B^2}{2\kappa}$：

$$A = -\bar{r}(\tau - B) + \frac{s^2}{2\kappa^2}(\tau - B) - \frac{s^2 B^2}{4\kappa} = \Big(\bar{r} - \frac{s^2}{2\kappa^2}\Big)(B - \tau) - \frac{s^2 B^2}{4\kappa}.$$

√ 这就是第 28 课中的公式。

练习 28.7

手算。 取 $g = x^4 + c_1 x^2 + c_2$：$g_t = c_1' x^2 + c_2'$，$g_{xx} = 12x^2 + 2c_1$。方程 $g_t + \tfrac{1}{2} g_{xx} = 0$ 给出

$$(c_1' + 6) x^2 + (c_2' + c_1) = 0,$$

对每个 x 成立。所以 $c_1' = -6$，$c_2' = -c_1$，两者在 T 时都为零。因此 $c_1 = 6\tau$，$c_2 = 3\tau^2$，其中 $\tau = T - t$。

多项式，与模拟及方程对比。

```
import numpy as np

g = lambda t, x: x**4 + 6 * (T - t) * x**2 + 3 * (T - t) ** 2      # c1 = 6 tau, c2 = 3 tau^2


def residual(t, x, h=1e-3, k=1e-4):
    gt = (g(t + k, x) - g(t - k, x)) / (2 * k)
    gxx = (g(t, x + h) - 2 * g(t, x) + g(t, x - h)) / h**2
    return gt + 0.5 * gxx


rng = np.random.default_rng(287)
Z = rng.standard_normal(2_000_000)
print("    t     x      formula    simulated    PDE residual")
gaps = []
for t, x in ((0.0, 0.0), (0.0, 1.0), (0.5, 1.0), (0.5, -2.0), (0.9, 0.5)):
    sim = ((x + math.sqrt(T - t) * Z) ** 4).mean()
    gaps.append(abs(sim - g(t, x)) / max(g(t, x), 0.1))
    print(f"{t:5.1f} {x:5.1f}   {g(t, x):10.4f}   {sim:10.4f}    {residual(t, x):+.1e}")
```

它的输出。

```
    t     x      formula    simulated    PDE residual
  0.0   0.0       3.0000       3.0016    +1.0e-06
  0.0   1.0      10.0000      10.0168    +1.0e-06
  0.5   1.0       4.7500       4.7566    +1.0e-06
  0.5  -2.0      28.7500      28.7346    +1.0e-06
  0.9   0.5       0.2425       0.2428    +1.0e-06
```

读输出。 全部一致。在 $x = 0$、$t = 0$ 处，$g = 3$：又是正态分布的四阶矩（第 4 课），现在作为偏微分方程的解出现。

练习 28.8

用蒙特卡洛计算 Vasicek 债券，三种步长，两种积分。

```
kappa, r_bar, s, r0 = 0.5, 0.04, 0.01, 0.03


def formula(tau):
    B = (1 - math.exp(-kappa * tau)) / kappa
    A = (r_bar - s**2 / (2 * kappa**2)) * (B - tau) - s**2 * B**2 / (4 * kappa)
    return math.exp(A - B * r0)


rng = np.random.default_rng(288)
print("  maturity  formula  steps/yr   left error  trapezium error  left - trapezium")
errors = {}
for tau in (1, 2, 5, 10):
    for per_year in (10, 50, 250):
        n, h = tau * per_year, 1 / per_year
        x = np.full(100_000, r0)
        left = np.zeros(x.size); trap = np.zeros(x.size)
        for _ in range(n):
            new = x + kappa * (r_bar - x) * h + s * math.sqrt(h) *
rng.standard_normal(x.size)
            left += x * h
            trap += 0.5 * (x + new) * h
            x = new
        e_l, e_t = np.exp(-left).mean() - formula(tau), np.exp(-trap).mean() - formula(tau)
        errors[(tau, per_year)] = (e_l, e_t)
        print(f"{tau:10d}  {formula(tau):7.5f}  {per_year:8d}  {e_l:+11.6f}  {e_t:+15.6f}
{e_l - e_t:+16.6f}")
```

它的输出。

```
  maturity  formula  steps/yr   left error  trapezium error  left - trapezium
         1  0.96839        10    +0.000134        -0.000060         +0.000194
         1  0.96839        50    +0.000036        -0.000001         +0.000038
         1  0.96839       250    +0.000002        -0.000006         +0.000008
         2  0.93492        10    +0.000166        -0.000130         +0.000296
         2  0.93492        50    +0.000044        -0.000014         +0.000058
         2  0.93492       250    -0.000037        -0.000048         +0.000012
         5  0.83429        10    +0.000185        -0.000192         +0.000377
         5  0.83429        50    -0.000037        -0.000112         +0.000075
         5  0.83429       250    -0.000086        -0.000101         +0.000015
        10  0.68473        10    -0.000162        -0.000497         +0.000334
        10  0.68473        50    -0.000148        -0.000214         +0.000067
        10  0.68473       250    -0.000000        -0.000014         +0.000013
```

读输出。 左边两列的误差主要是抽样噪声，100,000 条路径时约为 0.0001。这掩盖了我们要找的效应。但两种积分使用同样的路径，所以它们的差（最后一列）几乎没有噪声。它与步长成正比：从每年 10 步到 50 步缩小 5 倍，到 250 步又缩小 5 倍。

为什么左端点和偏高。 利率从低于长期水平处出发，倾向于上升。左端点和使用较低的、较早的值，因而低估了 $\int r\,dt$，使债券过于值钱。这个效应的大小约为 $\frac{1}{2}h(E[r_T]-r_0)P$。取 $h=0.1$，期限 10：$E[r_T]=0.03993$，$P=0.6847$，所以约为 3.4e-04。测得的差为 3.3e-04。

自测

1. $g(0.5,1)=1+0.5=1.5$。

2. $g(t,X_t)$ 是鞅（由塔性质），而鞅的 dt 项为零。

3. rg 这一项：$g_t+ag_x+\frac{1}{2}b^2g_{xx}=rg$。

第 29 课解答

有限差分

练习 29.1 至 29.8 以及第 29 课自测题的解答。

练习 29.1

思路。 两个泰勒级数相减：偶数阶项相消。

第 1 步。

$$f(x+h) - f(x-h) = 2hf'(x) + \tfrac{1}{3}h^3 f'''(x) + \dots$$

第 2 步。 除以 $2h$：$\dfrac{f(x+h)-f(x-h)}{2h} = f'(x) + \tfrac{1}{6}h^2 f'''(x) + \dots$

前向差分保留了 $\tfrac{1}{2}hf''$ 项，所以它的误差是 h 的一阶。中心差分的误差是二阶：对小的 h 要小得多。

验证，取 $f = e^x$ 在 0 处，$h = 0.1$：

- 中心：误差 0.001668；$\tfrac{1}{6}h^2 = 0.001667$。
- 前向：误差 0.05171；$\tfrac{1}{2}h = 0.05000$。

练习 29.2

思路。 两个泰勒级数相加：奇数阶项相消。

$$f(x+h) + f(x-h) = 2f(x) + h^2 f''(x) + \tfrac{1}{12}h^4 f''''(x) + \dots$$

减去 $2f(x)$，再除以 h^2：$f'' + \tfrac{1}{12}h^2 f'''' + \dots$

验证。 对 e^x 在 0 处，$h = 0.1$：误差 0.000834；$\tfrac{1}{12}h^2 = 0.000833$。

练习 29.3

思路。 用链式法则换元，然后取差分，再按邻点合并。

第 1 步。 $V(t,S)=U(\tau,y)$，其中 $y=\ln S$，$\tau=T-t$。所以 $V_t=-U_\tau$，$V_S=U_y/S$，$V_{SS}=(U_{yy}-U_y)/S^2$。

第 2 步：代入。 $-U_\tau+rU_y+\frac12\sigma^2(U_{yy}-U_y)=rU$。记 $\nu=r-\frac12\sigma^2$：

$$U_\tau=\nu U_y+\tfrac12\sigma^2U_{yy}-rU.$$

第 3 步：离散化。 对 τ 用前向差分，对 y 在旧时刻用中心差分，贴现项用 rU_i^{n+1}：

$$(1+r\Delta\tau)U_i^{n+1}=U_i^n+\nu\Delta\tau\frac{U_{i+1}^n-U_{i-1}^n}{2\Delta y}+\tfrac12\sigma^2\Delta\tau\frac{U_{i+1}^n-2U_i^n+U_{i-1}^n}{\Delta y^2}.$$

第 4 步：合并。

- U_{i+1}^n：$p_u=\Delta\tau\big(\frac{\sigma^2}{2\Delta y^2}+\frac{\nu}{2\Delta y}\big)$。
- U_{i-1}^n：$p_d=\Delta\tau\big(\frac{\sigma^2}{2\Delta y^2}-\frac{\nu}{2\Delta y}\big)$。
- U_i^n：$p_m=1-\frac{\sigma^2\Delta\tau}{\Delta y^2}$。

第 5 步：它们之和为 1。 $p_u+p_d=\sigma^2\Delta\tau/\Delta y^2$，它抵消了 p_m 的第二项。√

练习 29.4

思路。 一个波在这个格式下保持形状，只是被乘以一个因子。这个乘数决定一切。

第 1 步。 取 $\nu=r=0$，$p_u=p_d=c/2$，$p_m=1-c$。代入 $U_i^n=A^ne^{\mathrm{i}i\xi\Delta y}$，再除以 $A^ne^{\mathrm{i}i\xi\Delta y}$：

$$A=\tfrac{c}{2}e^{\mathrm{i}\xi\Delta y}+1-c+\tfrac{c}{2}e^{-\mathrm{i}\xi\Delta y}=1-c+c\cos(\xi\Delta y).$$

第 2 步。 利用 $1-\cos\phi=2\sin^2(\phi/2)$：$A=1-2c\sin^2(\xi\Delta y/2)$。

第 3 步：取值范围。 随着 ξ 变化，$\sin^2$ 从 0 取到 1，所以 A 从 1 降到 $1-2c$。要对每个波都有 $|A|\le1$，需要 $1-2c\ge-1$，即 $c\le1$。

最坏的波满足 $\sin^2=1$：$\xi\Delta y=\pi$，即第 29 课的锯齿，$A=1-2c$。

用文字说。 当 $c\le1$ 时，新值是三个邻点的真正平均，权重非负。平均值不能超过最大的邻点，所以什么也不会增长。当 $c>1$ 时，$p_m<0$：格式减去了中间值，这放大了差异，锯齿就自我强化。

练习 29.5

思路。 把工作量和误差都用 M 表示。

网格。 $\Delta y \propto 1/M$。稳定性要求 $\Delta\tau \propto \Delta y^2$，所以 $N \propto M^2$。工作量 $= MN \propto M^3$。误差 $\varepsilon \propto \Delta y^2 \propto M^{-2}$。所以 $M \propto \varepsilon^{-1/2}$，工作量 $\propto \varepsilon^{-3/2}$。

蒙特卡洛。 误差 $\propto$ 工作量$^{-1/2}$，所以工作量 $\propto \varepsilon^{-2}$。

哪个胜出。 当 $\varepsilon \to 0$ 时，$\varepsilon^{-3/2}$ 比 ε^{-2} 增长得慢。对单只股票，网格在每个有用的精度上都胜出，而且要求的精度越高，胜得越多。在 d 维中，网格的工作量像 $\varepsilon^{-(d+2)/2}$ 那样增长，所以在 $d = 2$ 时两者的速率持平，维数更高时蒙特卡洛胜出（第 30 课）。

练习 29.6

思路。 中心差分需要两侧各有一个邻点，而边缘点只有一侧。

看涨期权。 远低于行权价时，期权实际上不可能到期处于实值：$V \approx 0$。远高于行权价时，它必定被行权，所以就是一份远期：$V \approx S - Ke^{-r\tau}$。

看跌期权。 是镜像。远低于时，它必定被行权：$V \approx Ke^{-r\tau} - S$。远高于时，$V \approx 0$。

要延伸多远。 $\ln S_T$ 的标准差为 $\sigma\sqrt{T} = 0.2$。延伸到 ± 5 个标准差的网格从约 36.8 到 272。终止于它之外的概率约为 10^{-6}，所以边缘处的任何误差对 S_0 处的影响都可以忽略。

练习 29.7

显式网格上的欧式和美式看跌期权，行权价在网格点上。

```
import numpy as np


def put_grid(M, N, american):
    dy = 10 * sigma * math.sqrt(T) / M
    gap = math.log(K / S0)
    dy = gap / max(1, round(gap / dy))                        # strike on a grid point
    y = math.log(S0) + dy * (np.arange(M + 1) - M // 2)
    dt = T / N
    a, b = dt * sigma**2 / (2 * dy**2), dt * nu / (2 * dy)
    s = np.exp(y)
    pay = np.maximum(K - s, 0)
    v = pay.copy()
    for n in range(1, N + 1):
        new = v.copy()
        new[1:-1] = ((a + b) * v[2:] + (1 - 2 * a) * v[1:-1] + (a - b) * v[:-2]) / (1 + r *
dt)
        new[0] = K - s[0] if american else K * math.exp(-r * n * dt) - s[0]
        new[-1] = 0.0
        if american:
            new = np.maximum(new, pay)
        v = new
    return float(v[M // 2])


bs_put = o["bs"]["put"]
print("     M       N    European     error    American     error from 9.478")
eu_err = []
for M, N in ((100, 1000), (200, 4000), (400, 16000)):
    e, a = put_grid(M, N, False), put_grid(M, N, True)
    eu_err.append(e - bs_put)
    print(f"{M:6d}  {N:6d}   {e:8.4f}   {e - bs_put:+.5f}   {a:8.4f}   {a - 9.478:+.5f}")
print(f"error ratio, last two grids: {eu_err[1] / eu_err[2]:.2f}")
```

它的输出。

```
     M       N    European     error    American     error from 9.478
   100    1000     9.0117   -0.01318     9.4623   -0.01573
   200    4000     9.0228   -0.00203     9.4750   -0.00300
   400   16000     9.0243   -0.00051     9.4767   -0.00128
error ratio, last two grids: 4.00
```

读输出。 在最后两个网格之间，欧式误差缩小 4.00 倍：Δy 上的二阶。美式看跌期权趋近约 9.477。它的精度大致相同，但不那么规则，因为提前行权边界引入了第二个折点，而它每一步都在移动。

练习 29.8

M = 200 时的三种格式，然后是 M = 800 时 N 翻倍带来的变化。

```
bs_call = o["bs"]["call"]


def thomas(lo, di, up, rhs):
    n = rhs.size
    c, d = np.empty(n), np.empty(n)
    c[0], d[0] = up / di, rhs[0] / di
    for i in range(1, n):
        m = di - lo * c[i - 1]
        c[i], d[i] = up / m, (rhs[i] - lo * d[i - 1]) / m
    x = np.empty(n)
    x[-1] = d[-1]
    for i in range(n - 2, -1, -1):
        x[i] = d[i] - c[i] * x[i + 1]
    return x


def scheme(M, N, w):
    # w = 0 explicit, w = 1 implicit, w = 1/2 Crank-Nicolson.
    half = 5 * sigma * math.sqrt(T)
    y = math.log(S0) + np.linspace(-half, half, M + 1)
    dy, dt = y[1] - y[0], T / N
    A, B = sigma**2 / (2 * dy**2), nu / (2 * dy)
    up, md, lo = A + B, -2 * A - r, A - B
    s = np.exp(y)
    v = np.maximum(s - K, 0)
    for n in range(1, N + 1):
        top = s[-1] - K * math.exp(-r * n * dt)
        rhs = v[1:-1] + (1 - w) * dt * (up * v[2:] + md * v[1:-1] + lo * v[:-2])
        rhs[-1] += w * dt * up * top
        v = np.concatenate([[0.0], thomas(-w * dt * lo, 1 - w * dt * md, -w * dt * up, rhs),
[top]])
        if np.abs(v).max() > 1e9:
            return float("nan")
    return float(v[M // 2])


print(f"{'N':>6} {'explicit':>11} {'implicit':>11} {'Crank-Nicolson':>15}   (errors, M =
200)")
for N in (50, 100, 200, 400, 1000):
    ex, im, cn = (scheme(200, N, w) - bs_call for w in (0.0, 1.0, 0.5))
    print(f"{N:6d} {'blown up' if ex != ex else format(ex, '+.5f'):>11} {im:+11.5f}
{cn:+15.5f}")

print("change when N doubles, M = 800 fixed:")
print(f"{'N':>6} {'implicit':>11} {'ratio':>6} {'Crank-Nicolson':>15} {'ratio':>6}")
prev, last = {}, {}
for N in (100, 200, 400, 800):
    row = {}
    for name, w in (("i", 1.0), ("c", 0.5)):
        row[name] = scheme(800, N, w) - scheme(800, 2 * N, w)
    ri = prev["i"] / row["i"] if prev else 0.0
    rc = prev["c"] / row["c"] if prev else 0.0
```

```
    show = (lambda x: f"{x:6.2f}") if prev else (lambda x: "      ")
    print(f"{N:6d} {row['i']:+11.6f} {show(ri)} {row['c']:+15.6f} {show(rc)}")
    prev, last = row, {"i": ri, "c": rc}
```

它的输出。

```
     N    explicit    implicit  Crank-Nicolson   (errors, M = 200)
    50    blown up    -0.01985        -0.00117
   100    blown up    -0.01056        -0.00123
   200    blown up    -0.00591        -0.00125
   400    +0.00509    -0.00358        -0.00125
  1000    -0.00032    -0.00219        -0.00125
change when N doubles, M = 800 fixed:
     N    implicit  ratio  Crank-Nicolson  ratio
   100   -0.004651              +0.000016
   200   -0.002327   2.00       +0.000004   4.00
   400   -0.001164   2.00       +0.000001   4.00
   800   -0.000582   2.00       +0.000000   4.00
```

读输出。 在 $N = 400$ 以下，显式格式爆炸。隐式和 Crank-Nicolson 都给出合理的答案。隐式误差随 N 缩小，直到达到空间误差；Crank-Nicolson 从一开始就处在那个空间误差上，约为 -0.00125。

阶。 为了把空间误差排除在外，在同一个 M 下比较各次运行：N 每翻倍带来的变化都只来自时间。隐式的变化减半（比值 2.00）：**一阶**。Crank-Nicolson 的变化变为四分之一（比值 4.00）：**二阶**。

自测

1. $N = \sigma^2 T / \Delta y^2 = 0.04 / 0.0001 = 400$。

2. 前向差分的误差与 h 成正比。中心差分和二阶差分的误差与 h^2 成正比。

3. 解一个三对角方程组。它很便宜，因为 Thomas 算法只需一次前向扫描和一次后向扫描，约 $3M$ 次运算。

第 30 课解答

选择方法

练习 30.1 至 30.8 以及第 30 课自测题的解答。

练习 30.1

思路。 对每种方法，用误差表示成本。

树。 $\varepsilon \propto 1/n$，所以 $n \propto 1/\varepsilon$，成本 $\propto n^2 \propto \varepsilon^{-2}$。

蒙特卡洛。 $\varepsilon \propto 1/\sqrt{M}$，所以 $M \propto \varepsilon^{-2}$，成本 $\propto \varepsilon^{-2}$。

所以对单只股票，两者以相同的速率增长。 决定胜负的是常数，而它极大地偏向树：在第 30 课中，125,250 个节点给出 0.0021 的误差，而 1,000,000 条路径的标准误差为 0.0126。而且树的平稳误差可以外推，这会改善它的速率。（显式网格更好：$\varepsilon^{-3/2}$。）

资产更多时。 d 种资产的树成本 $\propto n^{d+1}$，即 $\propto \varepsilon^{-(d+1)}$，增长得极其可怕。蒙特卡洛保持在 ε^{-2}。超过几种资产，它就没有竞争对手了。

练习 30.2

思路。 每多一种资产，点数乘以 100。

每种资产 100 个点、每点 8 字节的网格，一个时间步。

资产数	点数	GB
1	1.0e+02	8.0e-07
2	1.0e+04	8.0e-05
3	1.0e+06	8.0e-03
5	1.0e+10	8.0e+01
10	1.0e+20	8.0e+11

时间步。 在 $\pm 5\sigma\sqrt{T}$ 范围内每个方向 100 个点，$\Delta y = 10\sigma\sqrt{T}/100 = 0.02$。对一种资产，显式限制 $\sigma^2\Delta\tau/\Delta y^2 \le 1$ 给出 $\Delta\tau \le 0.01$，即 $N \ge 100$。对 d 种资产，限制大约紧 d 倍，因为每个方向都添加自己的一项。所以 $d = 3$ 时约需 300 步，$d = 5$ 时约需 500 步。

在笔记本电脑上。 内存大概 16 GB，$d = 4$ 放得下（0.8 GB），$d = 5$ 放不下（80 GB）。$d = 5$ 时每步 10^{10} 个点，时间无论如何也是承受不起的。所以在大约 $d = 4$ 或 $d = 5$ 时网格就不行了。在实践中，到 3 时就已经很勉强。

练习 30.3

思路。 数一数维数，然后寻找难点：提前行权、路径依赖、收益中的跳跃。

- **欧式看跌期权：** 公式。精确且即时。
- **单只股票的美式看跌期权：** 树或网格。一维，提前行权只需一行。
- **以 20 只股票平均值为标的的看涨期权：** 蒙特卡洛。二十个维度排除了网格。用几何平均或矩匹配的 Black-Scholes 值作控制变量。
- **在 80 处失效的单只股票障碍期权：** 网格，把边界恰好放在 80。（对连续监测有公式可用。蒙特卡洛可行，但必须修正步与步之间的穿越。）
- **5 只股票上的美式期权：** Longstaff-Schwartz。网格不行，而提前行权必须估计。
- **单只股票上数字期权的 delta：** 有公式就用公式。在路径上用似然比法。在树上用相邻节点也可以，但收益的跳跃使它们在行权价附近噪声很大。

练习 30.4

思路。 保持 Z 不变，把路径看作 S_0 的光滑函数。

第 1 步。 $S_T = S_0 G$，其中 $G = e^{(r-\frac{1}{2}\sigma^2)T+\sigma\sqrt{T}Z}$。所以 $\partial S_T/\partial S_0 = G = S_T/S_0$。

第 2 步。 $\max(S_T - K, 0)$ 对 S_T 的导数是 $\mathbf{1}\{S_T > K\}$。由链式法则：

$$\frac{\partial}{\partial S_0}\left[e^{-rT}\max(S_T - K, 0)\right] = e^{-rT}\mathbf{1}\{S_T > K\}\frac{S_T}{S_0}.$$

第 3 步：它的平均值。 利用第 20 课的股票项，$\tilde{E}[e^{-rT}S_T\mathbf{1}\{S_T > K\}]/S_0 = S_0 N(d_+)/S_0 = N(d_+)$。

为什么折点没关系。 收益是连续的，只是它的导数在一个点上跳跃，而那个点的概率为零。它对 S_0 的导数在几乎每条路径上都存在，且斜率有界。所以导数的平均就是平均的导数。

练习 30.5

思路。 导数可以放在密度上，而不是放在收益上。

为什么路径法失效。 数字期权的收益在 K 的两侧都是平的，所以沿几乎每条路径，它的导数都是 0。这些导数的平均是 0，而真正的 delta 并不是。所有的敏感性都在跳跃处，逐路径求导根本看不到它。这里平均与求导不能交换。

似然比。

第 1 步。 写 $V = e^{-rT}\int f(s)\,p(s;S_0)\,ds$，其中 p 是 S_T 的对数正态密度。对 S_0 的唯一依赖在 p 中。

第 2 步：在积分号下求导，并乘以再除以 p：

$$\frac{\partial V}{\partial S_0} = e^{-rT}\int f(s)\frac{\partial_{S_0} p}{p}p\,ds = e^{-rT}\tilde{E}\Big[f(S_T)\frac{\partial \ln p}{\partial S_0}\Big].$$

第 3 步：权重。 $\ln p = -\dfrac{(\ln s - \ln S_0 - (r-\frac{1}{2}\sigma^2)T)^2}{2\sigma^2 T} + (\text{不含 } S_0 \text{ 的项})$。求导得

$$\frac{\partial \ln p}{\partial S_0} = \frac{\ln s - \ln S_0 - (r-\frac{1}{2}\sigma^2)T}{\sigma^2 T\,S_0} = \frac{\sigma\sqrt{T}Z}{\sigma^2 T S_0} = \frac{Z}{S_0\sigma\sqrt{T}}.$$

所以 delta 估计就是收益乘以 $Z/(S_0\sigma\sqrt{T})$。它不需要 f 的导数，所以任何收益都可以。

练习 30.6

思路。 一个你实际能遵循的规则，不可能胜过最优规则。

偏低的估计。 真实的美式价值是所有不窥视未来的行权规则的上确界。一个拟合出的规则一旦固定，就是这样的一个规则。把它用在与拟合无关的新路径上，规则不会窥视它们的未来，它的平均值是该规则价值的无偏估计。而这个价值不可能超过上确界。所以平均而言，估计值等于或低于真实价值。

为什么重复使用路径会把它推高。 在拟合用的路径上，回归已经看到了每条路径自己的未来现金流。所以规则在一定程度上是为那条路径的未来量身定做的，带有少量的先见之明。行权决策从中获益，这就是偏高的偏差。它通常很小，因为回归的参数少而路径多，而且它部分抵消了来自不完美规则的偏低偏差。

练习 30.7

Longstaff-Schwartz，用不同的路径拟合和估值。

```
import numpy as np


def simulate(paths, dates, gen):
    h = T / dates
    return S0 * np.exp(((r - 0.5 * sigma**2) * h + sigma * math.sqrt(h) *
gen.standard_normal((paths, dates))).cumsum(axis=1))


def fit_rule(paths_s, dates, degree):
    # Work backwards on the fitting paths; keep one polynomial per exercise date.
    df = math.exp(-r * T / dates)
    cash = np.maximum(K - paths_s[:, -1], 0.0)
    coefs = [None] * dates
    for i in range(dates - 2, -1, -1):
        cash *= df
        itm = K - paths_s[:, i] > 0
        x = paths_s[itm, i] / K
        c = np.polyfit(x, cash[itm], degree)
        coefs[i] = c
        ex = K - paths_s[itm, i]
        do = ex > np.polyval(c, x)
        cash[np.where(itm)[0][do]] = ex[do]
    return coefs


def value(paths_s, dates, coefs):
    # Follow the fitted rule on fresh paths: exercise the first time it says so.
    df = math.exp(-r * T / dates)
    n_p = paths_s.shape[0]
    pv = np.zeros(n_p)
    alive = np.ones(n_p, bool)
    for i in range(dates - 1):
        s = paths_s[:, i]
        go = alive & (K - s > 0)
        go[go] = (K - s[go]) > np.polyval(coefs[i], s[go] / K)
        pv[go] = (K - s[go]) * df ** (i + 1)
        alive &= ~go
    pv[alive] = np.maximum(K - paths_s[alive, -1], 0) * df**dates
    return pv.mean(), pv.std(ddof=1) / math.sqrt(n_p)


tree = o["am"]["tree"]
print(f"tree, exercise at any time: {tree:.4f}")
print("degree  dates   value      se     gap from the tree")
res = {}
for degree in (1, 2, 3, 5):
    for dates in (12, 50):
        gen = np.random.default_rng(307 + degree)
        coefs = fit_rule(simulate(100_000, dates, gen), dates, degree)
        v, se = value(simulate(200_000, dates, gen), dates, coefs)
```

```
        res[(degree, dates)] = v
        print(f"{degree:6d}  {dates:5d}   {v:7.4f}  {se:.4f}   {v - tree:+.4f}")
```

它的输出。

```
tree, exercise at any time: 9.4776
degree  dates   value      se     gap from the tree
     1     12    9.3529  0.0215   -0.1247
     1     50    9.3406  0.0207   -0.1370
     2     12    9.4100  0.0206   -0.0675
     2     50    9.4322  0.0199   -0.0453
     3     12    9.4434  0.0206   -0.0341
     3     50    9.4657  0.0200   -0.0119
     5     12    9.4142  0.0203   -0.0633
     5     50    9.4547  0.0201   -0.0229
```

读输出。 除了抽样噪声，所有值都低于树，正如练习 30.6 所说的那样。多项式的次数最要紧：直线显然太粗糙（50 个日期时为 9.341），而三次达到 9.466。升到 5 次并没有帮助（9.455）：多出的项拟合的是噪声。规则好的时候，更多的行权日有帮助：对三次多项式，从 12 个日期的 9.443 升到 50 个日期的 9.466。

练习 30.8

数字期权的 delta，用扰动法和似然比法。

```
dm = (math.log(S0 / K) + (r - 0.5 * sigma**2) * T) / (sigma * math.sqrt(T))
disc = math.exp(-r * T)
exact = disc * phi(dm) / (S0 * sigma * math.sqrt(T))
rng = np.random.default_rng(308)
M = 1_000_000
Z = rng.standard_normal(M)
grow = np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)
print(f"exact delta of the cash digital: {exact:.5f}")
print("    bump    estimate      se     paths that differ")
out_se = {}
for b in (5.0, 1.0, 0.1, 0.01):
    d = disc * (((S0 + b) * grow > K).astype(float) - ((S0 - b) * grow > K).astype(float)) /
(2 * b)
    out_se[b] = d.std() / 1000
    print(f"{b:8.2f}   {d.mean():8.5f}  {out_se[b]:.5f}   {(d != 0).sum():8d}")
lr = disc * (S0 * grow > K) * Z / (S0 * sigma * math.sqrt(T))
print(f"likelihood ratio: {lr.mean():.5f}  se {lr.std() / 1000:.5f}")
```

它的输出。

```
exact delta of the cash digital: 0.01900
    bump    estimate      se     paths that differ
    5.00    0.01874  0.00004     193133
    1.00    0.01901  0.00009      39188
    0.10    0.01904  0.00030       3925
    0.01    0.01868  0.00095        385
likelihood ratio: 0.01903  se 0.00003
```

读输出。扰动法面临两难。大的扰动有偏：取 5 时，差商是对一个宽范围内的 delta 取平均。小的扰动噪声大：取 0.01 时，只有几百条路径落在两个数字期权之间的小窗口里，每条贡献一个巨大的量 $1/(2b)$。标准误差像 $1/\sqrt{b}$ 那样增长：扰动为 1 时为 0.00009，为 0.01 时为 0.00095。

似然比法没有扰动，没有偏差，标准误差为 0.00003，精确值为 0.01900。应该用的就是它。

自测

1. 它需要约 $100^{10} = 10^{20}$ 个点。光是内存就不存在。

2. 因为价格差与任一估计的噪声相比都极小。用同样的随机数，几乎所有噪声在相减时都抵消了。用不同的随机数，噪声会叠加。

3. 似然比法。路径法在几乎每条路径上都给出 0，因为除了跳跃处，收益都是平的。